

Prague University of Economics and Business
Faculty of Informatics and Statistics



Developing a Serious Game to Enhance Prevention and Phishing Awareness

BACHELOR THESIS

Academic program: Applied Informatics

Author: Denys Serkes

Supervisor: Bc. et Bc. Jaroslav Svoboda, MSc

Prague, May 2025

Acknowledgement

I want to thank Bc. et Bc. Jaroslav Svoboda, MSc, for his guidance, support, and feedback throughout the writing of this thesis. Also, I want to thank all the testers for their valuable feedback.

Abstract

The primary objective of this thesis is to develop a serious game aimed at raising awareness about phishing and educating users on how to avoid it. The theoretical part explores the concept of serious games, examining their definition, effectiveness, and successful applications. An initial game concept was developed and used to identify and analyse similar games, which informed the creation of the final concept. This final concept includes a detailed description of the game's storyline, visuals, functionalities, and levels. A list of features was compiled and prioritised using the MoSCoW method, distinguishing between those implemented during the thesis and those reserved for future development.

The practical part outlines the game's development process, beginning with a description of the chosen tools—specifically the Unity game engine and the C# programming language. Development was divided into two phases: a low-fidelity prototype and a high-fidelity prototype. The low-fidelity prototype involved designing the user interface, implementing game functionality, and creating basic visuals. The high-fidelity prototype focused on refining visuals and improving the interface.

After development, the game underwent two types of evaluation. First, its visual and functional quality was assessed through user interviews. Testers provided feedback on what they liked, disliked, and suggested improvements. Second, the game's educational effectiveness was evaluated through a questionnaire consisting of eight phishing-related questions. Each participant completed the questionnaire both before and after playing the game. The results, presented in a bar chart, showed a 33.33% increase in correct responses on the second attempt, highlighting the game's potential as an effective educational tool. Finally, all user feedback was analysed, and the identified improvements were implemented in the final version of the game.

Keywords: serious game, Unity, phishing

Abstrakt

Hlavním cílem této práce je vývoj edukační hry (serious game), jejímž účelem je zvýšit povědomí o phishingu a vzdělávat uživatele, jak se mu vyhnout. Teoretická část se zabývá konceptem edukačních her, přičemž zkoumá jejich definici, účinnost a úspěšné příklady použití. Na základě počátečního konceptu hry byly identifikovány a analyzovány podobné hry, což vedlo k vytvoření finálního návrhu. Tento návrh obsahuje podrobný popis příběhu, vizuální stránky, funkcionality a jednotlivých úrovní hry. Vlastnosti hry byly seřazeny a upřednostněny pomocí metody MoSCoW, která rozlišuje funkce implementované v rámci této práce a funkce plánované do budoucna.

Praktická část popisuje proces vývoje hry, počínaje výběrem nástrojů – konkrétně herního engine Unity a programovacího jazyka C#. Vývoj probíhal ve dvou fázích: vytvoření low-fidelity prototypu a high-fidelity prototypu. V rámci první fáze byl vytvořen uživatelský interface, veškeré funkcionality hry a jednoduché vizuály. Druhá fáze se zaměřila na zpřesnění vizuální stránky a uživatelského rozhraní.

Po dokončení vývoje byla hra podrobena dvěma typům hodnocení. Nejprve byla prostřednictvím rozhovorů s uživateli posouzena její vizuální a funkční kvalita. Testeři sdělili, co se jim líbilo, nelíbilo a co by doporučili zlepšit. Dále byla ověřena vzdělávací efektivita hry pomocí dotazníku obsahujícího osm otázek souvisejících s phishingem. Každý účastník vyplnil dotazník před a po odehrání hry. Výsledky, znázorněné sloupcovým grafem, ukázaly zvýšení počtu správných odpovědí o 33,33% při druhém pokusu, což naznačuje potenciál hry jako efektivního vzdělávacího nástroje. Na závěr byla analyzována veškerá zpětná vazba od uživatelů a navržená vylepšení byla implementována do finální verze hry.

Klíčová slova: serious game, Unity, phishing

Contents

Acronyms

Introduction	1
1 Basic terms	5
1.1 Non-Player character	5
1.2 Game mechanics	5
1.3 Gameplay	5
2 What are Serious games	7
2.1 Term and explanation	7
2.2 Gamification vs SG	7
2.3 Why use it in the first place?	8
2.4 Serious Game examples	9
3 Pre-production	13
3.1 Initial concept	13
3.2 What is phishing?	13
3.3 Analysis of similar games	14
3.3.1 Interland	14
Kind Kingdom	14
Reality River	15
Mindful Mountain	16
Tower of Treasure	17
3.3.2 Hacknet	18
3.4 Final concept	19
3.4.1 Main idea	19
3.4.2 Story	19
3.4.3 Visuals	19
3.4.4 Gameplay	20
3.4.5 MoSCoW	21
4 Production	23
4.1 Working tools	23
4.1.1 Game engine	23
4.1.2 GitHub	23
4.2 Licence	24
4.3 Dividing the development process	25
4.4 Lo-fi prototype	26
4.4.1 UI	27
Introduction UI	27

	Hint test and Guess test UI	27
	Guess test with timer	28
	Score result	28
	Test UI	29
4.4.2	Introduction to story	30
4.4.3	Hint test	32
	Score	32
	Teacher introduction	34
	PopupManager	34
	SlideManager	36
	Suspicious elements gathering	40
4.4.4	Guess test	43
	Updating Score & UI	43
	Handling player choices	43
	Timer	44
	Updating the timer	45
4.4.5	Score result	47
4.5	Hi-fi prototype	47
4.5.1	Visuals	47
	Colour purpose	48
4.5.2	Text font	49
4.5.3	Asset preparation	50
4.5.4	Background creation	51
	Introduction to the story, Score result	51
	Pop-up window	53
	Hint test and Guest test	53
4.5.5	Teacher lines and phishing examples	55
5	Testing	57
5.1	Sharing the game	57
5.2	Feedback	57
5.2.1	Gameplay and visuals	58
	Tester 1	58
	Tester 2	58
	Tester 3	59
5.2.2	Main goal accomplishment	59
6	Pre-launch	61
6.1	Feedback analysis	61
6.2	Change font	62
6.3	Show the number of items the player still needs to click on	62
6.4	Add a recap of the level's goal	63
6.5	Highlight the element that was already clicked on	65
6.6	Modify pop-up navigation	65

6.7	Penalties for mistakes	66
6.8	Sound feedback	66
6.8.1	Finding the sounds	67
6.8.2	Click sound	67
6.8.3	Success sound and Failure sound	68
6.8.4	Background music	69
Conclusion		71
References		73
Appendices		77

List of Figures

2.1	<i>Hungry Red Planet</i> gameplay representation [25]	10
2.2	<i>FreeDive</i> gameplay representation [6]	11
3.1	Level King Kingdom gameplay representation [18]	15
3.2	Level Reality River gameplay representation [18]	16
3.3	Level Mindful Mountain gameplay representation [18]	17
3.4	Level Tower of Treasure gameplay representation [18]	17
3.5	<i>Hacknet</i> gameplay representation [53]	18
4.1	Example of a phishing email from the game (source: author)	29
4.2	Illustration of a pop-up at the Lo-fi stage (source: author)	36
4.3	Example of the pop-up navigation at Lo-Fi stage (source: author)	40
4.4	Example of <i>Guess test</i> level when the player runs out of time (source: author)	47
4.5	Illustration of a colour palette used to create visuals (source: author)	48
4.6	Roboto-Bold font example [19]	49
4.7	Valorax font example [14]	50
4.8	Introduction to story level visuals representation (source: author)	52
4.9	Score result level visual representation (source: author)	52
4.10	Pop-up visual representation (source: author)	53
4.11	Hint test visuals representation (source: author)	54
4.12	Guess test visuals representation (source: author)	54
5.1	Bar graph representing testers' results in the questionnaire (source: author)	60
6.1	Venite Adoremus font representation (source: author)	62
6.2	Level's goal recap visual representation (source: author)	64

List of Listings

4.1	File indicating license (source: author)	24
4.2	README file code (source: author)	24
4.3	Code responsible for loading the next scene (source: author)	31
4.4	Code responsible for managing the score (source: author)	32
4.5	Code responsible for managing pop-up status (source: author)	34
4.6	Method responsible for initialising all the variables inside the SlideManager script (source: author)	36
4.7	Method responsible for updating pop-up buttons' status (source: author) . .	37
4.8	Method responsible for switching the slides (source: author)	38
4.9	Method responsible for creating the SlideManager instance (source: author) .	39
4.10	Method responsible for updating the status of a suspicious element after the player's click (source: author)	40
4.11	Method responsible for managing suspicious element status and updating score (source: author)	41
4.12	Method responsible for updating pop-up status after player finishes the Hint test level (source: author)	42
4.13	Method responsible for updating pop-up status after player finished <i>Guess test</i> (source: author)	43
4.14	Method responsible for showing correct pop-up after player finishes <i>Guess test</i> level (source: author)	44
4.15	Method responsible for updating timer (source: author)	45
4.16	Methods responsible for starting, stopping, and restarting the timer (source: author)	46
4.17	Method responsible for showing final score result to the player (source: author)	47
6.1	Code responsible for initialising variables used for suspicious elements counter (source: author)	63
6.2	Code responsible for updating suspicious elements counter status (source: author)	63
6.3	Code responsible for updating the Sprite Renderer of the respective suspicious element (source: author)	65
6.4	Code responsible for disabling the btnHideIntro (source: author)	65
6.5	Code responsible for loading the next scene (source: author)	65
6.6	Code responsible for decreasing the score after the player runs out of time (source: author)	66
6.7	Code responsible for managing sounds for buttons (source: author)	67
6.8	Code responsible for playing respective sounds after the player finishes <i>Guess test</i> level (source: author)	69
6.9	Code responsible for playing background music (source: author)	70

Acronyms

GDD Game Design Document.

GO Game object.

Hi-fi High-fidelity.

Lo-fi Low-fidelity.

NPC Non-Player character.

SG Serious games.

UI User Interface.

Introduction

Every day, people open emails that look trustworthy - some of them end up losing personal data or even money. Phishing attacks are based on user inattention and a lack of knowledge and continue to evolve. Although many people believe they could spot a phishing attempt, real-world results often prove otherwise. Traditional methods of teaching cybersecurity, such as lectures or brochures, tend to be unengaging and have limited long-term impact. An alternative is to use serious games: games designed not just for fun, but to educate. They can offer memorable interactive experiences that help users learn through practice. This thesis explores the concept of serious games and presents the development of a game aimed at helping players recognise and avoid phishing.

The main goal of the thesis

The main goal of this thesis is to create a game that explains the concept of phishing and teaches players how to detect it. Here are the sub-goals of the thesis:

1. Explore the concept of serious games to gain a better understanding of the topic.
2. Analyse different examples of serious games.
3. Use the analysis from the previous chapter to develop the concept of the game.
4. Create a game using C# programming language and Unity Game Engine.
5. Test the game.
6. Use feedback from testers to improve the game.

Game Life Cycle

Such subgoals were chosen because each game goes through the seven steps of the game development process, which is called the *Game Life Cycle* [26]. Newman considers that the Game Life Cycle contains only five steps, while Petro Prots from the game development company Stepico defines some extra steps to describe the whole cycle with greater precision [27]. These are seven steps:

- Planning – This stage focuses on research and selecting crucial aspects of the game. Key tasks include identifying the target audience, conducting market research, assembling a team, and creating a budget.
- Pre-production – By the end of this stage, the team should have a *Game Design Document* (GDD). This document contains detailed information about the game's functionality, world, and main story.
- Production – This stage covers the entire process of creating the game. Developers work on game functionality, sound design, characters, levels, and other essential elements.

-
- Testing – Testers actively play the game to identify and report bugs. The team then prioritizes and fixes these issues to ensure a smooth gaming experience.
 - Pre-launch – At this stage, most of the game is complete, and developers focus on polishing the final product. The team also starts marketing and promotional efforts to raise awareness and generate interest in the game.
 - Launch – This is when the game is officially released to the public. Developers typically fix critical bugs and gather user feedback to improve the game.
 - Post-production – In this final stage, developers continue to support the game after release. They analyse user feedback, release updates, and introduce new content to enhance the gaming experience.

All these stages are essential for developing a successful game. However, since this game is not being created for commercial purposes, its lifecycle has been modified to align with the primary goal of this thesis. Below is the final structure of the thesis:

1. What are Serious games? – This chapter focuses on research on the topic of serious games to understand why it exists and find successful examples.
2. Pre-production – This chapter uses the knowledge about the serious games gained from previous chapter to create initial concept, use it to find similar games and create the final concept.
3. Production – This chapter takes the final concept from the previous chapter and uses it to create the game.
4. Testing – Describes the testing phase, including how feedback was gathered from testers.
5. Pre-launch – Explains the improvements made based on tester feedback.

Methods

This thesis will describe the game's development from the initial concept to the finished product. In the beginning, the author will conduct research on Serious Games using available documents. Then, he will create an initial concept and improve it based on the analysis of similar games using methods of observation, comparison, and analysis of available documents. The initial concept will contain information about the future development plan, which was created using the MoSCoW method. The author will use open sources.

Using the final concept, the author will develop a game using his prior knowledge, official Unity documentation, and video tutorials. The development process will be divided into Low-Fidelity and High-fidelity prototypes. After completing the development process, the author will present his game to a group of testers and gather feedback using the methods of observation and interviews.

Additionally, the author will conduct a knowledge assessment using questionnaires to test the game's effectiveness in increasing phishing awareness. Testers will complete a phishing-related quiz before playing the game and again after finishing it. The results will be gathered

and compared to evaluate the game's impact on the players' understanding of the phishing problem.

In the end, the game will be improved based on the testers' feedback and the knowledge assessment results.

1. Basic terms

The thesis contains various terms related to game development. To avoid confusion among readers, it is necessary to explain them first.

1.1 Non-Player character

A Non-Player character (NPC) is a character in a game controlled by the computer. The player can interact with the NPC for various reasons, such as progressing through the story or receiving a valuable reward. NPCs can take multiple forms: a monster, a friendly villager, a bandit, etc [55].

1.2 Game mechanics

Game mechanics are the set of rules and processes that define how a game works. They determine how the user plays the game, the outcomes of certain actions, and the intended way the game should be played. For example, in football, to score a point, a player needs to move the ball using their feet into the opponent's goal [2].

1.3 Gameplay

Gameplay is the overall experience created through the interaction between the player and the game's mechanics, storyline, and challenges. It defines how the player feels and engages with the game [13].

2. What are Serious games

This chapter explores the concept of serious games. It includes research on their definition, analyses of successful examples, and evaluation of their effectiveness.

2.1 Term and explanation

The definition of Serious games (SG) is quite broad, and many games that serve educational purposes do not explicitly identify as SGs. One of the most well-known descriptions of SGs was provided by Clark C. Abt in his book *Serious Games*. He defines serious games as those that “have an explicit and carefully thought-out educational purpose and are not intended to be played primarily for amusement” [7].

Even though this statement was made nearly 50 years ago, it remains highly relevant. However, it is important to note that not every SG “has a carefully thought-out educational purpose”. Some games are designed purely for entertainment, and their creators may not have intended any educational benefits. However, these games can become educational when used in a different context [7]. For example, many people play chess simply for enjoyment, but at the same time, it can be used to enhance mathematical problem-solving skills [34].

2.2 Gamification vs SG

There is another term which is commonly confused with SG, and it is called Gamification.

Kai Huotari and Juho Hamari, in their conference paper *Defining Gamification: A Service Marketing Perspective*, define gamification as “a process of enhancing a service with affordances for gameful experiences in order to support the user’s overall value creation” [20]. Simply put, gamification refers to the application of game mechanics to enhance user engagement and improve process efficiency.

Nico King, the executive director of Chaos Theory Games, which develops SG, describes the main difference between gamification and SG as follows: “Gamification adds to an experience, while serious gaming disrupts traditional experiences” [23].

Gamification does not change how or why a task is performed; it makes the process more fun and engaging. In contrast, SG changes the entire structure of the process—the task itself and the way it is performed can be entirely different from the initial concept [23].

For example, Duolingo is a mobile app whose primary goal is to teach people different languages. It uses various game mechanics (such as scores, rewards, and leaderboards) to make

the learning process more engaging. However, it does not provide a fundamentally different way to learn a language. Instead, it uses game functionality to enhance the experience [23].

On the other hand, *Minecraft: Education Edition* is a game that teaches students various subjects in a non-traditional way. It transforms the learning experience by changing how material is presented, using game functionality to create a smoother and more interactive learning process [23].

2.3 Why use it in the first place?

Now that we have a basic understanding of SGs, the question arises: why should we use them in education? Are they more efficient as educational tools than traditional teaching methods? Do they offer greater engagement, improved knowledge retention, and better learning outcomes than lectures, textbooks, or other conventional approaches?

The efficiency of SGs is a controversial topic due to a lack of sufficient research and evidence. This gap is largely attributed to the primary audience of SGs: children. Several obstacles hinder the widespread adoption and validation of SGs, including the time teachers need to familiarise themselves with the games, the difficulty of persuading stakeholders of their effectiveness, and challenges in aligning SG content with school curricula [31].

Nevertheless, there is evidence supporting the benefits of SGs and games in general as educational tools. Nico King from Chaos Theory Games highlights several advantages of SGs over traditional teaching methods. First, SGs have been shown to result in higher retention rates compared to other learning approaches. They also offer clear objectives by breaking down challenges into smaller, manageable goals. One of the most significant benefits is cost-effectiveness — SGs allow for the simulation of real-world scenarios, enabling organisations to save on training expenses. For example, PIXO VR is a game which simulates construction work. It helps construction companies deliver safety training to employees through digital simulations, reducing the need for costly real-world setups [22].

On the other hand, Breuer and Bente see different reasons why serious games are more effective than traditional teaching methods.

If we compare two seemingly different activities—playing chess and learning a new language—we can identify significant similarities. Chess has a defined set of rules that players must understand before playing. A player must invest considerable time practising and competing against others to become proficient. As a result, mastering chess is a long and challenging process [7]).

Similarly, learning a new language is also a lengthy and complex endeavour. Each language has a structured set of grammatical and syntactical rules that learners must grasp to achieve proficiency. At first glance, both processes share many common characteristics. However, an important question arises—why does playing chess often feel more engaging and enjoyable

than learning a new language [7]?

From the Breuer and Bente perspective, the interactivity which games offer is the crucial difference between these two activities. This interaction can happen on different levels:

1. On a micro-level of individual inputs and outputs. For example, the player pushes the button and the character attacks an enemy.
2. On a narrative level. For example, the player can manipulate the story to get a different ending.
3. On a meta-level of manipulating the game's rules. For example, players can modify the game by changing the difficulty or adding different modifications to it [7].

All these types of interaction allow players to manipulate the game and instantly receive feedback for their actions. This sense of control motivates players to explore the game further [7]. At the same time, during language learning, receiving feedback requires first mastering basic vocabulary and grammar. Only after reaching this stage can learners experience input, such as successfully using the language.

On the other hand, chess, for example, requires only an understanding of the rules to start playing. From the very beginning, players receive constant feedback—moving pieces, capturing opponents' pieces, and strategising their next moves. And this is just a board game. Video games take interactivity even further, using elements like music, storytelling, and interactions with the game world and characters, making the experience even more engaging.

2.4 Serious Game examples

In theory, SGs appear to be an effective educational tool, but does this hold true in practice? As mentioned in the previous chapter, the effectiveness of SGs remains a subject of debate. Nevertheless, many SGs have proven their value and practical applications in real-world scenarios. Here is a list of different SGs.

America's Army

America's Army is a first-person shooter video game released in 2002. The game provides a highly realistic representation of combat scenarios, with weapons and vehicles modelled after real-world examples [31].

Originally, *America's Army* was developed as a marketing tool to attract volunteers to the U.S. Army. The initiative proved to be highly successful, as the game contributed to recruiting soldiers at a rate of 15% compared to other recruitment programs. By autumn 2004, the game had a community of 4 million players, and each month it was increasing by 100 000 players. Additionally, it significantly increased public awareness of the Army—30% of Americans aged 16 to 24 reported learning about the military through the game [31].

Beyond recruitment, the game also served as a pre-training tool for new soldiers, providing them with a basic understanding of combat situations. Moreover, with certain extensions, active-duty soldiers even utilised it to prepare for real-life missions [31].

Hungry Red Planet

Hungry Red Planet was developed by Health Media Lab. The player takes on the role of a space explorer on a mission to colonise Mars. To succeed, they must establish a healthy, food-producing diet for the NPCs [10].

To win, players learn how to interpret FDA Nutrition Facts labels on 300 real foods available for menu planning and must align their menus with the Department of Agriculture’s Food Guide Pyramid and Healthy Eating Index. In addition, they need to plan food production and allocate resources to expand their settlements. If they keep the settlers healthy and produce enough food, they succeed. The game’s primary goal is to teach children how to maintain a healthy diet and avoid junk food [10].



Figure 2.1: *Hungry Red Planet* gameplay representation [25]

FreeDive

FreeDive is an example of a game that helps pediatric patients distract themselves from painful treatments. It was developed in partnership with the Believe in Tomorrow National Children’s Foundation [6].

In this game, the player takes on the role of a diver exploring the ocean. They can discover various sea creatures and search for hidden treasures, making the experience engaging and immersive for young patients [6].



Figure 2.2: *FreeDive* gameplay representation [6]

3. Pre-production

This chapter includes an introduction to the game’s initial concept, an analysis of different SGs, and the development of the final concept.

3.1 Initial concept

The initial concept of the game outlines its core idea. It helps establish milestones that can be used to identify similar games for inspiration. Here are the main criteria for the game:

- The game will be developed for Windows and Mac.
- All text in the game will be written in English.
- The game should explain what phishing is and teach players how to detect it.
- The game should be simple and easy to play.

The last criterion is particularly important because it increases the number of potential players by ensuring that the game remains accessible and easy to understand, regardless of their prior knowledge or experience with similar games. By keeping the mechanics simple and intuitive, a wider audience, including seniors and those unfamiliar with the topic, will be able to benefit from the game.

3.2 What is phishing?

The game’s main theme is phishing, but why make a game about it? What is phishing, and is it really that serious of a problem?

Phishing is a type of cybercrime in which a fraud attempts to steal personal data by pretending to be someone else. For example, a criminal might call you and claim to be the bank manager of the bank you use. They may say that a suspicious transaction has occurred in your account and ask you to provide sensitive banking information. Phishing can be carried out in various ways, such as emails, phone calls, and social media messages [4].

Although phishing may not seem dangerous at first glance, it is the most common type of cybercrime. The APWG (Anti-Phishing Working Group) is an international organisation that fights various cybercrimes, especially phishing and identity theft. Their *Phishing Activity Trends Report* stated that they observed 989,123 phishing attacks in the fourth quarter of 2024. Here are more statistics from their report:

- In October 2024, 345,881 unique phishing website attacks were detected.
- In December 2024, 33,899 unique phishing email campaigns were detected.
- In November 2024, 333 brands were affected by phishing campaigns [5].

As you can see, phishing remains a significant issue, and criminals continue to use it to steal personal data. Initially, the author’s supervisor suggested creating a game about phishing. After conducting research, the author realised that phishing is still a major problem and saw a great opportunity to use the SG functionality to raise awareness about this issue.

3.3 Analysis of similar games

To create a new game, it is a good idea to first look at similar products to inspire and learn from them. Unfortunately, there are no good games on phishing, but the author found two games similar to the initial concept: *Interland* and *Hacknet*. Here is an analysis of them.

3.3.1 Interland

Interland is an adventure-packed online game which was developed and published by Google. The game teaches the fundamentals of digital safety and citizenship through hands-on practice [17].

Interland does not explicitly label itself as a SG. However, it serves as a strong example of how game mechanics can be used to teach cybersecurity concepts effectively.

The game consists of four different islands, each dedicated to a specific lesson and minigame. During these minigames, users encounter various slides explaining key topics related to digital safety. At the end of each level, players must complete a simple test to show their understanding of the material. Below is an analysis of each level.

Kind Kingdom

The main topic of the first island is online civility. At the beginning, the player sees several slides explaining the importance of being kind in the digital world and emphasising their responsibility to report troublemakers.

The main goal for this island is to “spread kindness”. The player must climb to the top of the level while collecting hearts and giving them to NPC. Additionally, the player can block “troublemakers” by stepping on a button that locks the enemy in a cage.

Throughout the game, the player earns points for various actions. However, the score does not affect the gameplay; it simply serves as a way to track progress.

At the end of each island, the player takes a simple test on the main topic of the island. Each test consists of three single-choice questions, each presenting a different scenario designed to assess the player’s understanding of the subject. For example, on this island, the test focuses on the importance of kindness on the Internet.

The final section of the level focuses on "curing" a troublemaker. To achieve this, the player must distribute hearts to multiple NPCs. With their support, the troublemaker eventually transforms into a regular NPC. This section aims to demonstrate that troublemakers are often individuals who have been hurt, and by showing kindness, the player can help them overcome their struggles and become better individuals.



Figure 3.1: Level King Kingdom gameplay representation [18]

Reality River

The main topic of this island is the importance of questioning information found on the Internet. The minigame for this level consists of a 10-question test, each presenting a different scenario. The user has three possible answers, but only one is correct. Each option represents a different action that the player could take in that situation. If the user selects the wrong answer, the game displays an explanation explaining why the choice was incorrect. For example, the first question asks: "You receive an email with a hilarious subject line that is obviously a scam." The user has three response options:

1. Delete it and forget about it.
2. Avoid clicking any links and mark them as "spam."
3. LOL. Forward it to your friends for a laugh, but tell them not to click anything.

If the user selects the third option, they fall into the river, and an explanation appears: "Never send a sketchy email to anybody, even for fun." If the user chooses the first option, they earn 15 points and proceed to the next question. Some questions include a timer that requires quick thinking.

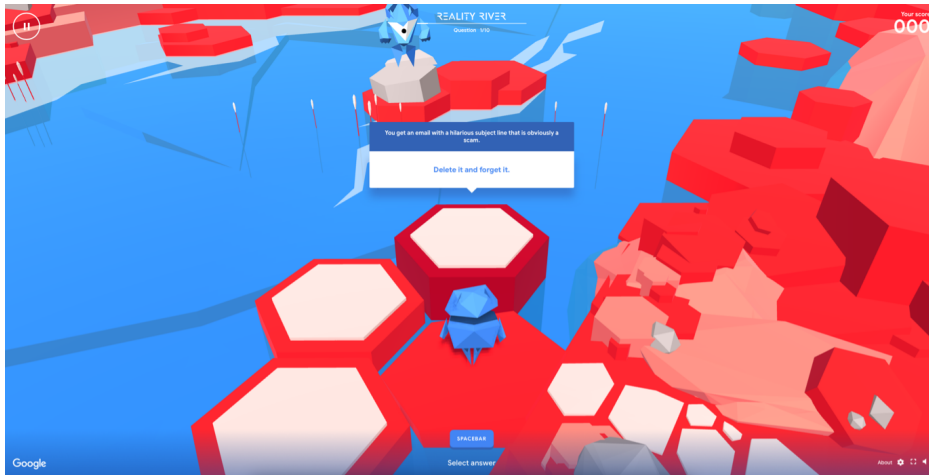


Figure 3.2: Level Reality River gameplay representation [18]

Mindful Mountain

The third island teaches players to keep an eye on their private information and to think twice before sharing it. On this island, the player interacts with a board filled with NPCs and mirrors. They control a machine that launches a beam of light, representing the exchange of data with NPCs. There are four types of NPCs:

- Blue – Parents.
- Green – Friends.
- Green – Friends.
- Yellow – Unknown people.
- Red – Troublemakers or cyber criminals.

The island consists of 10 different levels, the goal of each level being to direct the light so that it reaches only specific NPCs. To pass a level, the player must share data with at least one of the correct NPCs mentioned at the beginning. For each correct NPC, the player earns 10 points. However, if the light reaches the wrong NPC, the player loses 10 points per mistake and must restart the level. Here is an example level breakdown.

At the beginning of a level, the player is shown the type of information they are about to share: "A selfie of you and your best friend with her new haircut." The game then specifies with whom this information should be shared - in this case Green NPCs (Friends). The player must then position the machine correctly so that the light reaches only the Green NPCs without touching any other. If the light reaches an incorrect NPC, the player loses an attempt and a message appears explaining why the choice was incorrect.

The player can only pass the level by sharing the data with the correct NPCs. However, if they optimally position the machine, they can earn up to 50 points by using mirrors to reflect the light to additional Green NPCs.

After reaching level five, the game no longer specifies with whom the information should be

shared, requiring the player to decide independently based on the type of information.



Figure 3.3: Level Mindful Mountain gameplay representation [18]

Tower of Treasure

The last island focuses on securing personal data with strong passwords. At the beginning of the level, the player must reach a tower while collecting data along the way, such as passwords, photos, and messages. For each type of data collected, the player earns 5 points.

Upon reaching the tower, hackers begin to attack and the player must defend it by creating strong passwords. There are five gates and the player must choose three different passwords for each. Each time, the game provides tips on creating a strong password. If the player selects a weak password, he loses 10 points and receives a message explaining why the password is weak.



Figure 3.4: Level Tower of Treasure gameplay representation [18]

3.3.2 Hacknet

Hacknet is a 2D hacker simulator where players use a terminal to steal data from various computers. It was developed by Team Fractal Alligator and published by Fellow Traveller in 2015 [53]. Although Team Fractal Alligator does not classify the game as serious, it incorporates educational elements that can serve as inspiration.

The game follows a storyline that begins with a message from a hacker named Bit, the creator of the world's most invasive security system. Unfortunately, Bit has died, and after failing to reconnect to his system for 14 days, an automated failsafe is activated, sending instructions via email to the player. Throughout the game, the player follows Bit's instructions in an attempt to delete Hacknet-OS.

Hacknet simulates a Unix-like operating system, which requires players to hack into various computers to steal or delete data. Although the game does not explicitly teach hacking, it introduces educational aspects by familiarising players with Unix shell commands. On the right side of the screen, the user has access to a command-line interface that functions as a real terminal. Players must use Unix shell commands such as `rm`, `cd`, and `ssh` to progress through the game.

The game provides an immersive experience with an engaging storyline that offers the sensation of being a hacker. Additionally, by requiring players to interact with the terminal using real-life commands, the game serves as an entertaining way to learn the basics of command-line navigation.

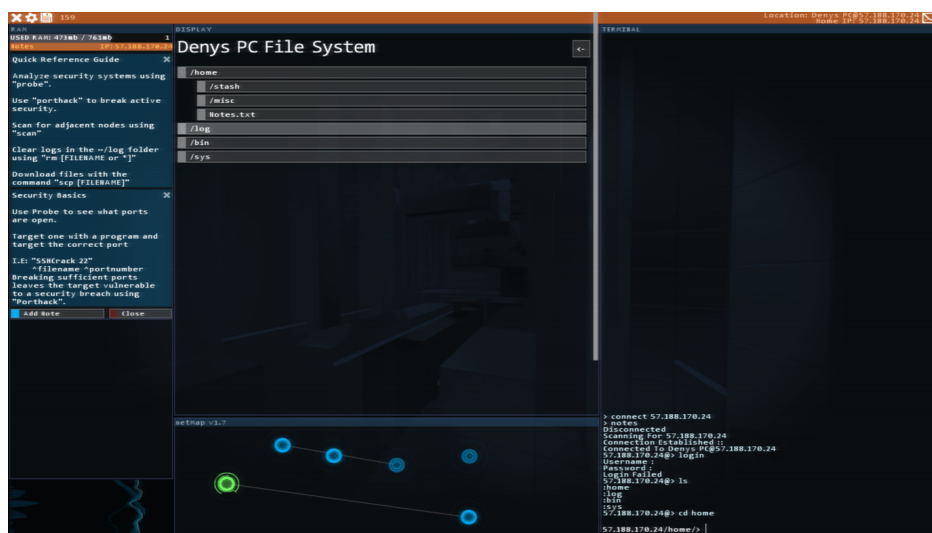


Figure 3.5: *Hacknet* gameplay representation [53]

3.4 Final concept

Now that the analysis of other SGs is complete, the final concept can be created. Usually, the development team creates a GDD at this stage of the Game Life Cycle.

The GDD is a document that contains all essential information about the game, allowing anyone to access it and understand what it is about. Typically, it includes details about the story, characters, mechanics, levels, and visuals [11].

The structure of the GDD was divided into 5 parts:

1. main idea,
2. story,
3. visuals,
4. gameplay,
5. MoSCoW.

3.4.1 Main idea

The name of the game is Cyber Academy. The game will be in 2D. The main idea behind the game is to teach players how to recognise phishing and raise awareness of this issue.

3.4.2 Story

The game takes place in a distant future, in the year 2358. In 2132, all people were transferred to the metaverse due to a cataclysm, so now everybody lives in the digital reality. The digitalised world gave immortality and freedom to people, but not everyone uses these benefits with good intentions. Metaverse, where everything is digitalised, frauds, and hackers have become the top threat.

The only ones who can stand against them are cyber-detectives. You are a student at the Cyber Academy and your dream has always been to become a cyber-detective. The player will see examples of phishing emails from the far past and he needs to learn what phishing is and how to fight against it.

3.4.3 Visuals

The visuals of the game should be simple and create an atmosphere of a distant future. All visual elements, such as the User Interface (UI), font and background elements, should contribute to this ambience. The main inspiration for designing the UI and other visual elements was Hacknet, as it creates a similar atmosphere.

3.4.4 Gameplay

The game will consist of 6 levels. At the beginning of each level, the player will see a pop-up window representing the teacher's communication with the player. At the beginning of each level, the teacher will introduce the main goal and topic.

One possible way to improve this mechanic is by adding a voiceover for the teacher's dialogue. This addition would enhance immersion, making the game world feel more engaging, and would also increase accessibility for players with visual impairments. At this stage of development, implementing such a feature is not feasible, as high-quality voice recording requires specific equipment. However, this functionality could be added in future updates.

After the explanation, the player will have the opportunity to apply their knowledge through interactive exercises. After completion of the level, the teacher will provide feedback on any mistakes. *Interland* mainly inspired this game functionality. With this mechanic, the player will learn about phishing more quickly by receiving immediate feedback, allowing them to recognise mistakes in real time and understand how to avoid similar threats in the future.

The player will have a score system that tracks their progress. Making correct choices throughout the game will increase the score, while incorrect choices will decrease it. *Interland* also inspired this mechanic. The score should not overwhelm the player, and that is the main reason why it does not affect anything. It is simply used to show the player's growth and progression.

Below is a detailed breakdown of each level:

- Introduction to the story - The player is introduced to the story of the game. The main goal of this level is to explain the story and what is the main goal of the game. After the player has finished reading, he can press a button to switch to the next level.
- Hint test 1 – The teacher explains the concept of phishing, provides an example, and shares tips on how to identify it. The player is then presented with a phishing example and must click on suspicious elements that indicate that it is a phishing. Each correctly identified element awards 5 points. Once the player thinks that he has found all the suspicious elements, he clicks a button to complete the test. At the end of the test, he receives feedback on any missed elements, with explanations of why they are suspicious. If all elements are correctly identified, the player receives a congratulatory message from the teacher.
- Hint test 2 – This level follows the same mechanics as *Hint test 1*, but the teacher introduces urgency-based phishing, and the player must analyse an example of this type.
- Guess test 1 – In this level, the player is not informed if the scenario is phishing or not. Using the knowledge gained from previous lessons, they must analyse the scenario and make a decision. The player is given two options: "This is phishing" and "This is not phishing". Choosing the correct answer awards 10 points and provides an explanation

of the example. Selecting the wrong option reduces the player's score by 10 points.

- Guess test 2 – This level follows the same mechanics as Guess Test 1 but introduces a time limit. If the player fails to make a decision before the timer runs out, they are prompted to try again.
- Score result – this level thanks the player for playing the game and shows their final score.

3.4.5 MoSCoW

The MoSCoW method is a four-step approach that divides project functionality into four parts based on its impact on the final product. Each game has a defined budget and deadline, so the MoSCoW method helps define the priority of specific mechanics, levels, or story elements. The term MoSCoW stands for “Must have, Should have, Could have, and Won't have” [8]. Here is the explanation of each step:

- Must have – This includes the required functionality necessary to release the game.
- Should have – These are important parts of the game with high priority, but not essential to finish the product.
- Could have – These features would be beneficial, but they do not significantly impact the final product.
- Won't have – This includes all functionality with the lowest priority, which the team may add after the game's launch.

The author plans to implement only the “Must have” and “Should have” features. The “Could have” and “Won't have” categories represent functionality that may be added in the future. Below is a list of all functionalities, categorised by priority:

Must have

- *Introduction to the story* level functionality,
- *Hint test* levels functionality,
- score,
- *Guess test* levels functionality,
- pop-up functionality.

Should have

- Timer functionality,
- *Score result* levels functionality.

Could have

- Different difficulty modes,
- additional levels covering more types of phishing,
- functionality to compare your score with other players,
- accessibility features to support players with various mobility impairments.

Won't have

- Release on IOS and Android platforms,
- voiceover feature for all NPCs.

4. Production

This chapter details the entire process of developing the game's functionality and visuals. It also covers the tools used for creating Low-fidelity (Lo-fi) and High-fidelity (Hi-fi) prototypes. In addition, it outlines the steps involved in building a game prototype based on GDD completed in the previous chapter.

4.1 Working tools

4.1.1 Game engine

To start developing a game, it is necessary to choose a game engine. A game engine is a framework or a system designed to create video games, typically including essential tools such as a level editor, animations, audio, and UI editors [33].

Steam is one of the largest PC digital game distributors, developed by Valve Corporation [54]. SteamDB is an independent third-party website that tracks information about the games and applications available on Steam [29]. In SteamDB, we can see that in 2025 the three most popular game engines are Unity, Unreal Engine, and GameMaker [30].

Unity was chosen as the game engine used to create this game because the author had the most experience with it and is also the most popular. This means that it has the most tutorials and assets on the Internet.

4.1.2 GitHub

GitHub is a web-based platform designed to host Git repositories and facilitate collaboration on software development projects. It builds on Git, a version control system, by providing additional tools that help developers manage their code, track changes, and work together more effectively. GitHub is widely used for open source and private projects, enabling teams of developers to work collaboratively [16].

The author chose GitHub over other web-based platforms because he had the most experience with it. He used it to track his progress and to keep a copy of the project in case of an emergency.

For the source code, see the GitHub repository. To play the game, you can check Cyber Academy webpage.

4.2 Licence

The MIT licence was chosen for the source code of the game. The MIT Licence is a permissive open source licence that allows anyone to freely use, modify, and redistribute the code with minimal restrictions, as long as the original license and copyright notice are retained [24]. This choice aligns with the educational and academic nature of the project, aiming to encourage broad reuse, collaboration, and accessibility. By using the MIT licence, it enables educators, students, and developers to easily build upon this work, including integrating it into their own projects, whether open-source or proprietary. It removes legal and technical barriers that might otherwise discourage adoption or experimentation. The licence file was added to the repository and was taken from the official MIT website [3].

```
1 # Released under MIT License
2
3 Copyright (c) 2025 Denys Serkes.
4
5 Permission is hereby granted, free of charge, to any person obtaining
   a copy of this software and associated documentation files (the "
   Software"), to deal in the Software without restriction, including
   without limitation the rights to use, copy, modify, merge,
   publish, distribute, sublicense, and/or sell copies of the
   Software, and to permit persons to whom the Software is furnished
   to do so, subject to the following conditions:
6
7 The above copyright notice and this permission notice shall be
   included in all copies or substantial portions of the Software.
8
9 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
   EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
   MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
   NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT
   HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
   WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
   OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
   DEALINGS IN THE SOFTWARE.
```

Listing 4.1: File indicating license (source: author)

The game includes third-party assets, which are described in later chapters. Most of these assets are licenced for personal use only, restricting redistribution and commercial use without explicit permission. As a result, the assets are not included in the Git repository. To enforce this, a `.gitignore` file was added. Additionally, a `README` file was created, which explains the chosen license for the code and provides links to the original assets in case someone wishes to continue developing the game using the same visuals.

```
1 # Cyber Academy - Bachelor thesis project
2
```

```
3 This is the source code for an serious game developed as part of my
   academic thesis. The project is designed to be open, reusable, and
   easy to build upon for educational or research purposes.
4
5 ## License
6
7 The source code is released under the [MIT License](./LICENSE), which
   allows free use, modification, and redistribution, even in
   commercial projects.
8
9 ## Assets Notice
10
11 **Note:** The game assets (such as images, audio, and fonts) are **not
   included** in this repository due to license incompatibilities.
   These assets are subject to different licenses that do not allow
   redistribution under the MIT License.
12
13 However, if you wish to continue developing the game using the same
   visuals, you can obtain the original assets from the following
   sources:
14
15 - Background sprites: [https://nyknck.itch.io/futuristicui]
16 - Teahcer sprite: [https://the-outlander.itch.io/free-pixel-art-
   cyberpunk-character-portraits-pack]
17 - Sound effects: [https://assetstore.unity.com/packages/audio/sound-fx
   /free-casual-game-sfx-pack-54116?srsltid=
   AfmB0oqT7Ljc7crpNmz7Ecuo7KnsRmEYJfmahUjEoehcPdZ5az1RKCR2]
18 - Background music: [https://assetstore.unity.com/packages/audio/
   ambient/sci-fi/free-sci-fi-and-cyberpunk-music-pack-264590]
19 - Fonts used for emails: [https://fonts.google.com/specimen/Roboto]
20 - Primary font: [https://www.dafont.com/venite-adoremus.font]
21 - Image used for user in email: [https://depositphotos.com/vector/
   unknown-person-silhouette-glasses-profile-picture-silhouette-
   profile-man-avatar-179308454.html]
22
23 Please ensure you **review and comply with the licenses** provided by
   each source if you intend to use these assets in your version of
   the game.
```

Listing 4.2: README file code (source: author)

4.3 Dividing the development process

The author decided to divide the game development process into two stages: the creation of a Lo-fi prototype and a Hi-fi prototype. The Lo-fi prototype serves as a quick method for transitioning the initial concept into a tangible product. In game development, Lo-fi focuses

on functionality rather than visuals. Basically, it tests whether the mechanics are engaging and allows for quick iterations and adjustments [28].

The Hi-fi prototype is an advanced iteration of the Lo-fi prototype, featuring more polished visuals and advanced functionality. It is developed once the developer clearly understands the final version of the product [28].

This approach helps prevent unnecessary time investment in visuals for mechanics that may be changed or removed later. Following this methodology, the author first created the game's functionality, UI, and basic visuals in the Lo-fi prototype. However, the Hi-fi prototype focused primarily on the visual aspects of the game.

4.4 Lo-fi prototype

The author started by creating an empty 2D project. Before diving into development, it is essential to explain three fundamental concepts in Unity: scenes, game objects, and components.

A scene represents a single level or section of the game. It contains all the content and the game functionality. A project can include multiple scenes, allowing developers to structure different parts of the game. For example, one scene might feature a cutscene in which the player interacts with a NPC, while another could focus on combat [41].

A Game object (GO) is an element within a scene. It serves as a building block for different game components and can take on various roles. For example, a GO can represent a player, an obstacle, or an enemy [43].

The component is a functional element that defines the behaviour of a GO. Developers use components to make GOs interactive. Unity provides many built-in components, such as the **Transform**, which controls an object's position and movement, and the **Sprite Renderer** component, which adds visuals to a game object. Each GO, by default, has a **Transform** component, so the author will not mention adding it later. Developers can also create custom components using C# [44].

Each scene includes a default GO called the **Main Camera**, which has two components: **Camera** and **Transform**. The **Camera** component acts as the player's viewpoint, rendering the scene, while the **Transform** component controls its position, rotation, and scale, determining how the player perceives the scene.

The following chapters describe the main goal of a single scene, the creation process, and specific aspects of Unity.

4.4.1 UI

The code itself has many manipulations with UI, so in the beginning it is necessary to understand the UI structure and how it was created.

Unity offers several ways to build a UI, but the most popular ones are Unity UI and UI Toolkit. Both have their own advantages, but during development, the author mainly used UI Toolkit for two main reasons:

1. UI Toolkit is rapidly evolving, with regular updates introducing new features that can be useful during the post-production phase [40].
2. The UI Toolkit is currently very popular, meaning plenty of up-to-date tutorials are available that align with the latest Unity versions.

To create UI using the UI Toolkit, an empty *UIDocument* must first be created. Then, a UXML file needs to be created and assigned to the *UIDocument* object. The UI can be designed in three ways:

- editing the UXML file directly,
- using C# scripting,
- utilising UI Builder.

Since UI was simple, UI Builder was chosen because it provided all the necessary functionalities.

In UI Builder, the UI is constructed using various components, such as `VisualElement`, `Label`, and `Button`. Each of these components has attributes similar to CSS properties, including margin, padding, display, position, and background.

Once the UI creation process is understood, we can examine the structure of each *UIDocument* in detail.

Introduction UI

The *UIDocument* for this scene is called `IntroToStory` and contains the following elements:

- `header Label` – Displays the text “History”.
- `introText Label` – Displays the story of the game.
- `btnNextScene Button` – Transitions the player to the next scene.

Hint test and Guess test UI

The *Hint test* UI and *Guess test* UI share several common elements, which will be explained first. Here is an explanation of the shared UI elements for *Hint test* and *Guess test*:

- **scoreLabel** – a **Label** element to display the score of the player.
- **IntroToLevelContainer** – a **VisualElement** which holds UI elements visible at the beginning of the level, when the introduction pop-up appears. It contains:
 - **SlidesIntro** – a **VisualElement** that holds **Label** elements, explaining the main goal of this level and the phishing concept.
 - **IntroBtnsContainer** – a **VisualElement** that holds the buttons to navigate through the introduction pop-up. It has **btnHideIntro**, **btnNextSlide** and **btnPrevSlide**.
 - **teacherName** – a **Label** which displays the teacher’s name.
- **PopUpExplanationContainer** – a **VisualElement** responsible for displaying the explanation pop-up after the player completes the level. It contains:
 - **successMsg** – a **Label** which displays a congratulatory message if the player catches all suspicious elements.
 - **SlidesExplanation** – a **VisualElement** that contains **Label** elements explaining each suspicious element in the scene.
 - **ExplanationBtnsContainer** – a **VisualElement** that holds buttons to navigate through the pop-up. It has buttons **btnNextSlide**, **btnPrevSlide** and **btnNextScene**.

The only difference between the *Hint test* UI and the *Guess test* UI are the buttons used to complete the test. In the *Hint test* UI, there is a **Button** element called **btnFinishTest**. While in the *Guess test* UI, instead of a single button, there is a separate **VisualElement** called **MainBtns**, which contains two **Button** elements called **btnIsPhishing** and **btnNotPhishing**.

Guess test with timer

The *Guess test* with a timer shares the same UI structure as other *Guess tests*, with the only difference being the additional elements used to display the timer logic.

- It includes a **Label** element called **timerLabel**, which displays the remaining time.
- It features a **VisualElement** called **FailTimerContainer**, which appears when the player fails to complete the test within the given time.

The **FailTimerContainer** follows the same structure as the **IntroToLevelContainer** and **PopUpExplanationContainer**, with one key difference. It contains only one **Button** element, **btnRestartTimer**, which allows the player to retry the test.

Score result

The score result section UI consists of three **Label** elements:

- **headerLabel** – a header that displays the text "Congratulations!"

- `textLabel` - is similar to `headerLabel` but displays the text "Thanks for playing!" with a different colour and font size.
- `scoreLabel` - displays the final score for the player.

Test UI

To add phishing email text to the scene, Unity UI was used. The author decided to use a different UI system for the following reasons:

- Overlap with other UI – The phishing text is positioned at the centre of the scene, causing it to overlap with other UI elements, making it difficult to work with. While the UI Toolkit uses the UI Builder to create UI elements, the Unity UI is created directly in the scene, preventing overlap with UI Toolkit elements.
- Prefab Support – UI created using Unity UI is a GO, allowing it to be saved as a *Prefab*, which simplifies the process of creating new levels.

Prefab is a saved GO with predefined properties that can be reused across multiple scenes [50].

To create UI elements with Unity UI, you first need to create a Canvas and add various elements to it, such as text, buttons, and images. To design the Test UI, the author added multiple text elements and images to make it look like an email.

To make the phishing example resemble a regular email, an image of a user was added to help the player feel that it is a real message. The author of the image states that it can be used freely for both commercial and non-commercial projects [9].

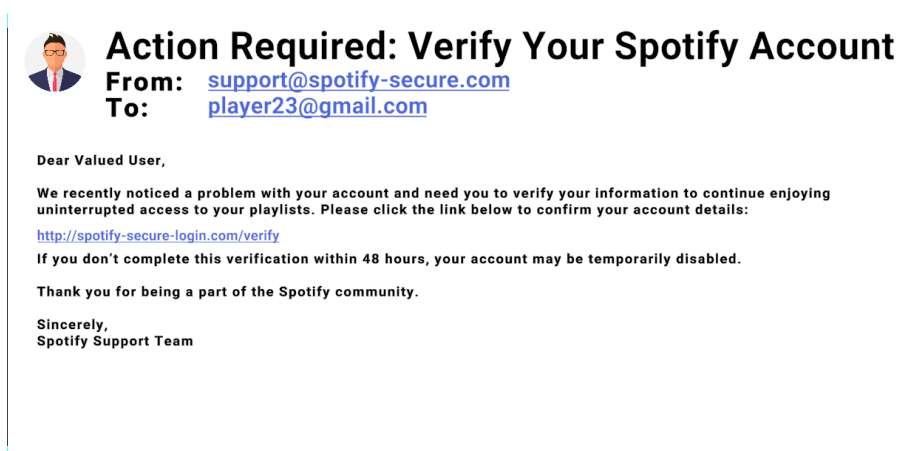


Figure 4.1: Example of a phishing email from the game (source: author)

4.4.2 Introduction to story

This is the beginning of the game, where the player is introduced to the story. It contains some text and a button that the player needs to click on to proceed to the next scene.

This scene has only one functionality: transitioning the player to the next scene after the button is clicked. The script responsible for this functionality is called **GameSceneManager**.

To create a script, we need right-click in the Project tab, select Create, and choose C# Script. This will generate a new public class with the provided name derived from **MonoBehaviour**.

MonoBehaviour is the base class for all Unity scripts. It provides access to Unity's core functionality and enables interaction with GOs and engine events. Scripts that derive from **MonoBehaviour** can utilise lifecycle methods such as **Start**, **Update**, and **OnDestroy**. It also helps to handle input, and communicate with other components. By default, all classes have two methods: **Start** and **Update** [46].

Start is a built-in Unity method that is automatically called once when the script is enabled. It runs before the first frame update. This method is usually used for initialising variables, setting up references to other objects, and performing actions that need to be ready when the game starts [48]. **Update** is also a built-in Unity method, but it is called every frame [49].

In the beginning, an array called **btnNextSceneArray** is created. Throughout the game, a single scene may contain multiple **btnNextScene** buttons, so it needs to reference all of them.

The **Start** method declares and initialises the variable **uiDocument**, which holds a reference to the first **UIDocument** in the scene. The variable **root** stores a reference to the main **VisualElement** inside this **UIDocument**. After that, it finds all **Button** elements within the **UIDocument** with the name **btnNextScene**, converts them into a list, and then into an array.

The reason why we need to first convert it to the list and only then to array, lies in how **Query<T>** works. It returns a **UQueryBuilder<T>**, an iterable structure allowing further filtering and modifications. However, **UQueryBuilder<T>** does not have a **ToArray()** method, but it does have **ToList()**. While it is possible to store **btnNextScene** as a list and avoid converting it into an array, **btnNextScene** is not meant to be modified dynamically. Using a fixed-size array improves performance since arrays are generally more efficient for static collections.

Finally, a **foreach** loop iterates through all buttons and subscribes the **OnClickNextScene** method to their click event.

OnClickNextScene is a method which is responsible for transitioning to the next scene. It uses **SceneManager** to get the index of the current scene, adds one to it, and then loads the scene with the updated index.

SceneManager is a Unity class responsible for managing scenes. The **SceneManager** class

allows to load, unload, and query scenes at runtime [51]. In this script:

- `SceneManager.GetActiveScene` retrieves the currently active scene.
- `SceneManager.LoadScene` loads a scene by its build index, transitioning the game to the specified scene.

```

1 public class GameSceneManager : MonoBehaviour
2 {
3     // Array to hold references to \texttt{Button} elements named "
4     btnNextScene"
5     private \texttt{Button}[] btnNextSceneArray;
6
7     private void Start()
8     {
9         // Find the UIDocument in the scene
10        var uiDocument = GameObject.FindObjectOfType<UIDocument>();
11        // Get the root element of the UI hierarchy
12        var root = uiDocument.root\texttt{VisualElement};
13
14        // Find all \texttt{Button} elements named "btnNextScene" and
15        store them in the array
16        btnNextSceneArray = root.Query<\texttt{Button}>("btnNextScene")
17        .ToList().ToArray();
18
19        // Assign the OnClickNextScene function to the "clicked" event
20        of each button
21        foreach (var btn in btnNextSceneArray)
22        {
23            btn.clicked += OnClickNextScene;
24        }
25
26        // Function to load the next scene
27        private void OnClickNextScene()
28        {
29            // Get the index of the current scene and increment it by 1
30            int nextSceneIndex = SceneManager.GetActiveScene().buildIndex
31            + 1;
32            // Load the scene at the calculated index
33            SceneManager.LoadScene(nextSceneIndex);
34        }
35    }

```

Listing 4.3: Code responsible for loading the next scene (source: author)

The last step is to activate the script in the scene. To do this, the script needs to be assigned as a component to a GO. `GameSceneManager` was assigned to a GO with the same name.

4.4.3 Hint test

The level following the introduction is the *Hint test*. As mentioned in the final concept, this level required several key functionalities, such as:

- score,
- suspicious elements gathering,
- teacher introduction pop-up,
- final explanation pop-up.

Score

The player's score increases when they choose the correct option. The script responsible for this logic is called **ScoreManager**.

After the introduction scene, the player needs to see his score in each scene, so the score value must persist across all scenes. For this purpose, **ScoreManager** was made as a singleton.

The Singleton pattern is a design pattern that ensures that there is only one instance of a class throughout the application [15]. This approach eliminates the need to create a separate **ScoreManager** in each scene while maintaining the score value across all scenes.

To verify that only one instance of the class exists, Unity's **Awake** method was used. **Awake** functions similarly to **Start**, which is called when the object is instantiated rather than before the first frame update. This ensures that **ScoreManager** becomes available as soon as possible.

Additionally, the built-in Unity method **DontDestroyOnLoad** prevents the object from being destroyed when a new scene loads, allowing it to persist across different scenes.

The **AddScore** method is responsible for managing the score and updating the UI. The **points** parameter determines the number of points to be added or deducted, as different actions in the game affect the score differently. To ensure that the score does not drop below zero, Unity's **Mathf** class is used.

Mathf is a Unity class that provides common mathematical functions, including rounding, trigonometric operations, and other utilities. The **Mathf.Max** method returns the larger of the two given values [45]. In this case, if the score drops below zero, it is set to 0.

Each scene has a unique **scoreLabel**, so whenever a new scene loads, a new **scoreLabel** must be initialised. This is done using the **RegisterLabel** method, which is called in the **Start** method of **GameSceneManager**, as this script is present in every scene.

```
1 public class ScoreManager : MonoBehaviour
2 {
```

```
3      // Singleton instance of ScoreManager, allowing global access to  
4      its methods and properties  
5      public static ScoreManager Instance { get; private set; }  
6      // Static variable to store the player's score, shared across all  
7      instances  
8      public int score = 0;  
9      // Static reference to the Label element used to display the score  
10     private Label scoreLabel;  
11  
12     private void Awake() {  
13         // Check if other instance exists  
14         if(Instance != null && Instance != this) {  
15             Destroy(gameObject);  
16             return;  
17         }  
18  
19         // Set this as the Singleton instance  
20         Instance = this;  
21         // Prevent this object from being destroyed when loading a new  
22         scene  
23         DontDestroyOnLoad(gameObject);  
24     }  
25     // Static method to add points to the player's score  
26     public void AddScore(int points)  
27     {  
28         // Increase the score by the given points, ensuring it doesn't  
29         go below zero  
30         score = Mathf.Max(0, score + points);  
31  
32         // Update the score Label element with the new score  
33         scoreLabel.text = $"Score:{score}";  
34     }  
35     // Method to initialize the Label element and update its value  
36     public void RegisterLabel(UIDocument uiDocument) {  
37         // Find the UIDocument in the scene and access the root UI  
38         element  
39         var root = uiDocument.root\texttt{VisualElement};  
40  
41         // Find the label named "scoreLabel" in the UI hierarchy and  
42         update the text value  
43         scoreLabel = root.Q<Label>("scoreLabel");  
44         scoreLabel.text = $"Score:{score}";  
45     }
```

44 }

Listing 4.4: Code responsible for managing the score (source: author)

Teacher introduction

At the beginning of the level, the player should see a pop-up displaying text. Since fitting a large amount of text into a single pop-up is impractical, the text is divided into multiple slides. Additionally, the player should have a navigation menu to switch between slides and close the pop-up to start the level.

Throughout the game, the player will encounter multiple pop-ups, so to make this system reusable, it was divided into two scripts: **PopupManager** and **SlideManager**.

PopupManager

The **PopupManager** script is responsible for managing the pop-up window. When the player clicks a button to finish the test, the following actions should occur:

- The test UI is hidden to prevent it from overlapping with the pop-up UI.
- **Buttons** in the scene are disabled and dimmed, so that the player can clearly see that they are inactive.
- The pop-up background is enabled.
- The explanation UI **VisualElement** is enabled.

The **SwitchPopup** method is responsible for performing mentioned steps. This method has four parameters:

- **elementName** – The name of the UI element that should be enabled or disabled. For example, if the player clicks a button to hide the introduction pop-up and start the level, this parameter will be set to the name of the **VisualElement** responsible for displaying that pop-up.
- **btnStatus** – The button's status should be set (enabled/disabled).
- **opacityElement** – The opacity value applied to buttons and other elements.
- **VisualElementStatus** – The status of the main **VisualElement**. If the value is true, the element will be enabled.

This method uses the given parameters to manipulate the background and UI status. Finally, it iterates through the **btnsToEnable** array, finds buttons with the specified names, and updates their status.

btnsToEnable is an array containing the buttons' names that can be enabled or disabled. It has the **SerializeField** attribute, which allows the variable's value to be edited inside Unity [52].


```
1 public void SwitchPopup(string elementName, bool btnStatus, float
   opacityElement, bool \texttt{VisualElement}Status)
2 {
3     // Switch the popup background status
4     popupBG.SetActive(!popupBG.activeSelf);
5
6     // Switch the popup UI status
7     canvas.enabled = !canvas.enabled;
8
9     // Update the scene info opacity(score, timer etc.)
10    sceneInfo.style.opacity = opacityElement;
11
12    // Visual element to be hidden or revealed
13    var elementSwitch = root.Q<\texttt{VisualElement}>(elementName
        );
14
15    if (\texttt{VisualElement}Status) {
16        elementSwitch.style.display = DisplayStyle.Flex;
17    }
18    else
19    {
20        elementSwitch.style.display = DisplayStyle.None;
21    }
22    // Toggle the opacity and active status of interactive buttons
        in the scene
23    foreach (var btnName in btnsToEnable)
24    {
25        var button = root.Q<\texttt{Button}>(btnName);
26        button.SetEnabled(btnStatus);
27        button.style.opacity = opacityElement;
28    }
29 }
```

Listing 4.5: Code responsible for managing pop-up status (source: author)

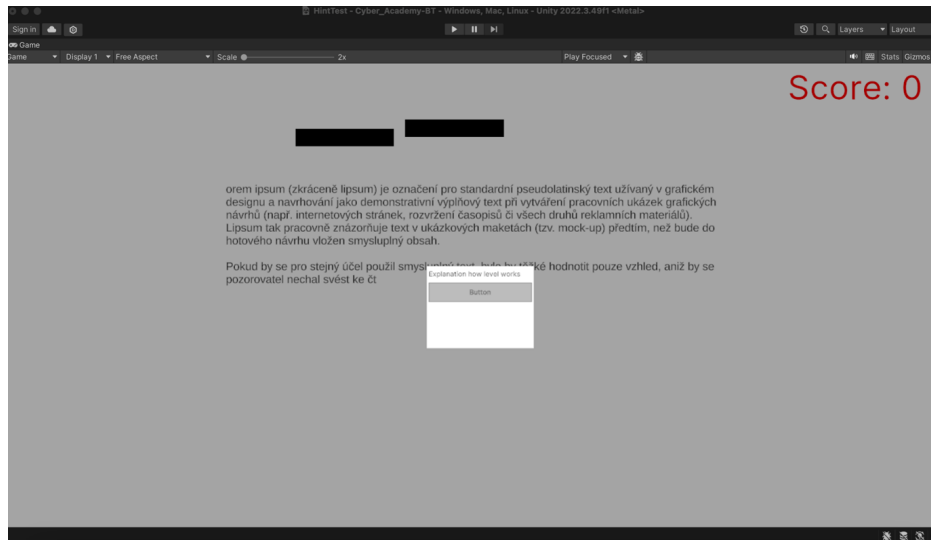


Figure 4.2: Illustration of a pop-up at the Lo-fi stage (source: author)

SlideManager

`SlideManager` is responsible for handling navigation between different slides.

The `Start` method initialises all the variables and disables buttons. It also checks the number of slides. If it is less than two, it sets the `isUpdateEnabled` variable to false. This variable is used inside the `Update` method.

```

1  private void Start()
2  {
3      // Find the UIDocument in the scene
4      var uiDocument = GameObject.FindObjectOfType<UIDocument>();
5
6      // Get the root element of the UI hierarchy
7      var root = uiDocument.root\texttt{VisualElement};
8
9      // Initialize Visual Element which holds all the slides
10     slideParentContainer = root.Q<\texttt{VisualElement}>(
        slideContainerName);
11
12     // Store all the slides to an array
13     slidesArray = slideParentContainer.Query<Label>().ToList().ToArray
        ();
14
15     // Disable Update method if there are less than two slides
16     if(slidesArray.Length < 2)
17     {
18         isUpdateEnabled = false;
19     }
20     //Initialize a Visual element with navigation buttons
21     btnsContainer = root.Q<\texttt{VisualElement}>(btnsContainerName);
22

```

```

23      // Initialize a \texttt{Button} element with name "btnNextSlide"
24      btnNextSlide = btnsContainer.Query<\texttt{Button}>("btnNextSlide"
25      );
26
27      // Assign the OnClickNextSlide method to the "clicked" to the
28      // button
29      btnNextSlide.clicked += OnClickNextSlide;
30
31      // Disable button by default
32      btnNextSlide.SetEnabled(false);
33
34      // Initialize a \texttt{Button} element with name "btnPrevSlide"
35      btnPrevSlide = btnsContainer.Q<\texttt{Button}>("btnPrevSlide");
36      btnPrevSlide.clicked += OnClickPrevSlide;
37      btnPrevSlide.SetEnabled(false);
38
39      // Initialize a \texttt{Button} element with name "btnHideIntro"
40      btnHideIntro = root.Q<\texttt{Button}>("btnHideIntro");
41      btnHideIntro.SetEnabled(false);
42  }

```

Listing 4.6: Method responsible for initialising all the variables inside the SlideManager script (source: author)

The `Update` method is responsible for controlling the state of the buttons. If the player is on the first slide, the button that displays the previous slide should be blurred and disabled. The same logic applies to the button displaying the next slide when the player is on the last slide.

To implement this logic, the variable `currentSlideIndex` is used to represent the index of the current slide.

- If `currentSlideIndex` equals 0, `btnPrevSlide` will be disabled; otherwise, it will be enabled.
- The same logic applies to `btnNextSlide`.
- If `isUpdateEnabled` is false, it means there is only one slide, so both `btnPrevSlide` and `btnNextSlide` should remain disabled

```

1  private void Update()
2  {
3      // Check if there are more than two slides
4      if(!isUpdateEnabled)
5      {
6          return;
7      }
8      // Enable btnPrevSlide if the current slide is not the first one
9      if(currentSlideIndex != 0)
10     {
11         btnPrevSlide.style.opacity = 1f;

```

```
12         btnPrevSlide.SetEnabled(true);
13     }
14     // Disable btnPrevSlide if the current slide is the first one
15     else if (currentSlideIndex == 0)
16     {
17         btnPrevSlide.style.opacity = .07f;
18         btnPrevSlide.SetEnabled(false);
19     }
20     // Disable btnPrevSlide if the current slide is the last one
21     if (currentSlideIndex == slidesArray.Length - 1)
22     {
23         btnNextSlide.style.opacity = .07f;
24         btnNextSlide.SetEnabled(false);
25     }
26     // Enable the btnNextSlide
27     else
28     {
29         btnNextSlide.style.opacity = 1f;
30         btnNextSlide.SetEnabled(true);
31     }
32 }
```

Listing 4.7: Method responsible for updating pop-up buttons' status (source: author)

OnClickNextSlide method is responsible for changing the display property of the slide at current index to None and showing the next slide. The same way works function OnClickPrevSlide.

```
1 // Method to switch to the next slide
2 private void OnClickNextSlide()
3 {
4     // Hide the current slide
5     slidesArray[currentSlideIndex].style.display = DisplayStyle.
        None;
6
7     // Increase the currentSlideIndex by one
8     currentSlideIndex ++;
9
10    // Reveal the next slide in the array
11    slidesArray[currentSlideIndex].style.display = DisplayStyle.
        Flex;
12 }
13
14 // Method to switch to the previous slide
15 private void OnClickPrevSlide()
16 {
17     // Hide the current slide
18     slidesArray[currentSlideIndex].style.display = DisplayStyle.
        None;
```

```

19
20     // Decrease the currentSlideIndex by one
21     currentSlideIndex --;
22
23     // Reveal the previous slide in the array
24     slidesArray[currentSlideIndex].style.display = DisplayStyle.
        Flex;
25 }

```

Listing 4.8: Method responsible for switching the slides (source: author)

`CreateSlideManager` method is responsible for creating a GO with the `SlideManager` script attached.

As mentioned earlier, each level contains two groups of slides: *Introduction* and *Explanation*. While the number of Introduction slides remains fixed, the number of Explanation slides can change dynamically during the level. Since `CreateSlideManager` creates the object itself and does not need to modify the state of the object, it is designed as a public static method. This approach eliminates the need to manually initialise `SlideManager` when calling the method from another script.

This allows it to be called in different scenarios, such as:

- The `Start` method of `HintObjectManager` creates a `SlideManager` instance for the introduction slides.
- When the player completes a test, it creates another `SlideManager` for the explanation pop-up.

```

1 // Creates a new SlideManager instance
2 public static void CreateSlideManager(string slideName, string
    btnContainer)
3 {
4     // Create a new Game object named "SlidesManager" to manage slides
5     GameObject slidesIntroManager = new GameObject("SlidesManager");
6
7     // Add the SlideManager component to the newly created Game object
8     SlideManager slideManagerScript = slidesIntroManager.AddComponent<
        SlideManager>();
9
10    // Assign the provided slide container name to the SlideManager
        script
11    slideManagerScript.slideContainerName = slideName;
12
13    // Assign the provided button container name to the SlideManager
        script
14    slideManagerScript.btnsContainerName = btnContainer;
15 }

```

Listing 4.9: Method responsible for creating the `SlideManager` instance (source: author)

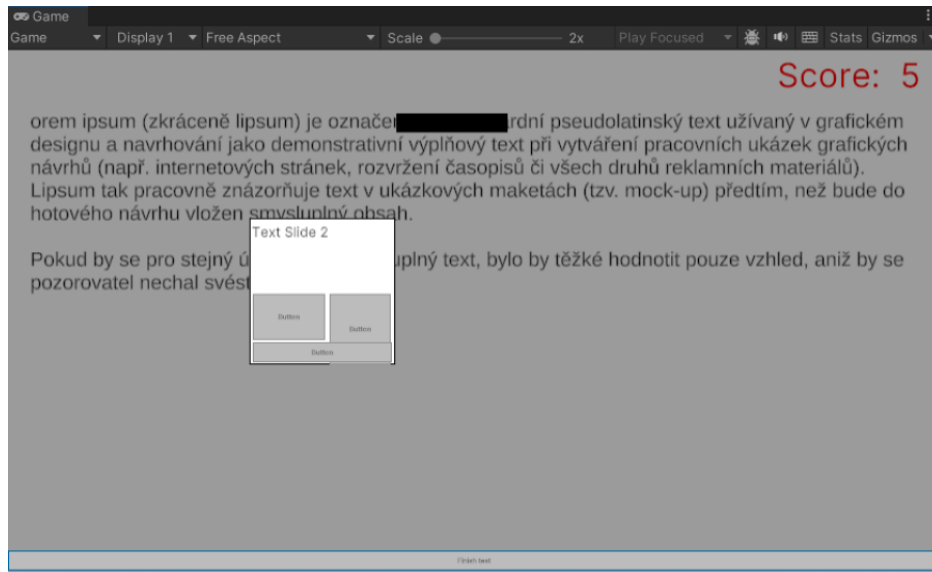


Figure 4.3: Example of the pop-up navigation at Lo-Fi stage (source: author)

Suspicious elements gathering

The player must click on suspicious elements to earn points. After completing the test, they click a button to finish and receive feedback. If any suspicious elements were missed, an explanation of those elements is shown; otherwise, a success message is displayed. The script responsible for managing suspicious elements is called `HintObjectManager`.

A separate GO represents each suspicious element and contains two components:

`Box Collider 2D` and `HintObject`.

- `Box Collider 2D` – Defines a rectangular collision area for the 2D object, enabling Unity to detect clicks on it [39].
- `HintObject` – Manages the logic associated with the suspicious element, such as interaction and scoring.

The author created the `HintObject` component. It uses a Unity built-in method called `OnMouseDown` to call a method from `HintObjectManager` called `OnClickHintObjectManager` when the player clicks on the object [47]. As a parameter, it passes the reference to itself.

```
1 private void OnMouseDown()  
2 {  
3     // Find the HintObjectManager in the scene.  
4     HintObjectManager hintManager = FindObjectOfType<HintObjectManager>  
5         >();  
6     // Call the OnClickHintObject method and pass the reference to  
7     clicked object  
8     hintManager.OnClickHintObject(gameObject);
```

8 }

Listing 4.10: Method responsible for updating the status of a suspicious element after the player's click (source: author)

The `OnClickHintObject` method is part of `HintObjectManager` and is responsible for handling player interactions with suspicious elements.

Each suspicious element has a unique `Label` element with the same name as its corresponding GO, explaining why the element is suspicious. When the player finishes the test, only the explanations for the missed elements should be displayed.

To achieve this functionality, a list called `hintObjects` was created. The `Start` method retrieves all suspicious elements in the current scene. Each of these elements is assigned a tag called `HintObject`. By using the built-in Unity method `FindGameObjectsWithTag`, all objects with this tag are collected and stored in the list.

A dictionary called `hintObjectDictionary` is then created, where:

- Key → Reference to the GO itself.
- Value → Reference to the corresponding `Label`.

A loop iterates through all objects in `hintObjects`, finds a `Label` with the same name, and stores the key-value pair in the dictionary.

At the beginning of the `OnClickHintObject` method, a variable is created to store the reference to the `Label` of the clicked suspicious element. The built-in Unity method `RemoveFromHierarchy` is then used to remove the corresponding `Label`, ensuring that the player does not see it when finishing the level. After that the object is removed from the dictionary and the `AddScore` method from `ScoreManager` is called to update the player's score.

```
1 public void OnClickHintObject(GameObject currentHintObject)
2 {
3     // Retrieve the associated slide from the dictionary using the
4     // clicked hint object reference
5     var explanationSlide = hintObjectDictionary[currentHintObject];
6
7     // Remove the respective slide element from the UI
8     explanationSlide.RemoveFromHierarchy();
9
10    // Remove the clicked hint object from the dictionary
11    hintObjectDictionary.Remove(currentHintObject);
12
13    // Update the score
14    ScoreManager.Instance.AddScore(5);
```

14 }

Listing 4.11: Method responsible for managing suspicious element status and updating score (source: author)

`OnClickFinishTest` is responsible for handling the completion of the test by:

- Hiding the test interface.
- Disabling `btnFinishTest`.
- Enabling the pop-up background.
- Displaying the first explanation slide.

At the beginning of the method, the length of the `hintObjectDictionary` is checked. If it equals zero, the player has successfully spotted all suspicious elements, and the `successMsg Label` is displayed. Otherwise, the first explanation slide is shown.

Since the first slide may vary depending on the player's performance, additional text is appended at the beginning of the slide. This message acknowledges the player's efforts while indicating that some elements were still missed.

The `SwitchPopup` method from `PopupManager` is then called to enable the pop-up background and disable other UI elements.

Finally, a new `SlideManager` object is created to allow navigation through the explanation slides.

```
1 private void OnClickFinishTest()
2 {
3     // Display the success message if no hint objects are left.
4     if (hintObjectDictionary.Count == 0)
5     {
6         successMsg.style.display = DisplayStyle.Flex;
7     }
8     else
9     {
10        // Retrieve the first slide, update the text value and reveal
            it
11        Label firstSlide = hintObjectDictionary.First().Value;
12        firstSlide.text = "Good Try! Now let's see what you missed.\n"
            + firstSlide.text;
13        firstSlide.style.display = DisplayStyle.Flex;
14
15        // Create new SlideManager for the explanation slides
16        SlideManager.CreateSlideManager("SlidesExplanation", "
            ExplanationBtnsContainer");
17    }
18
19    // Toggle the popup
20    var PopupManager = FindObjectOfType<PopupManager>();
```



```
21     PopUpManager.SwitchPopup("PopUpExplanationContainer", false, 0.07f
    , true);
22 }
```

Listing 4.12: Method responsible for updating pop-up status after player finishes the Hint test level (source: author)

4.4.4 Guess test

The `GuessTestManager` script is responsible for handling the *Guess test* in the game. It updates the score and modifies the text of the first slide based on the player's choice.

Updating Score & UI

The `ShowMessage()` method updates the score, modifies the first slide text, and enables the pop-up explanation.

```
1 private void ShowMessage(string msg, int points)
2 {
3     // Update the score value
4     ScoreManager.Instance.AddScore(points);
5
6     // Update the text value of the first slide
7     Label firstSlide = slidesExplanation[0];
8     firstSlide.text = msg + firstSlide.text;
9
10    // Create SlideManager for explanataion popup
11    SlideManager.CreateSlideManager("SlidesExplanation", "
    ExplanationBtnsContainer");
12
13    // Toggle the popup status
14    var PopUpManager = FindObjectOfType<PopUpManager>();
15    PopUpManager.SwitchPopup("PopUpExplanationContainer", false, 0.07f
    , true);
16 }
```

Listing 4.13: Method responsible for updating pop-up status after player finished *Guess test* (source: author)

Handling player choices

The methods `OnClickIsPhishing()` and `OnClickIsNotPhishing()` are triggered when the player selects an one of the two options to finish the test. `isPhishing` defines whether the displayed email is a phishing attempt. It also has the attribute `SerializeField`. `successMsg`

& `failMsg` are messages displayed on the first slide, depending on whether the player's choice was correct or incorrect. The player choice is unpredictable, so the first slide text should be updated to show the player whether he picked the right option or not.

Both methods have the same logic but pass different parameters to `ShowMessage()`, based on whether the email is actually phishing or not.

```
1 private void OnClickIsPhishing()
2 {
3     if(isPhishing)
4     {
5         ShowMessage(successMsg, audioClipWin, 10);
6     }
7     else
8     {
9         ShowMessage(failMsg, audioClipFail, -10);
10    }
11 }
12
13 private void OnClickIsNotPhishing()
14 {
15     if(isPhishing)
16     {
17         ShowMessage(successMsg, audioClipFail, -10);
18     }
19     else
20     {
21         ShowMessage(failMsg, audioClipWin, 10);
22     }
23 }
```

Listing 4.14: Method responsible for showing correct pop-up after player finishes *Guess test* level (source: author)

Timer

The `TimerManager` script manages the timer in the *Guess test*. It ensures that the countdown functions correctly, updates the UI, and handles timer-related events when the player runs out of time.

`TimerManager` has three important variables:

- `timeRemaining` - tracks how much time is left.
- `isTimerRunning` - indicates whether the timer is active.
- `timeLimit` - the initial amount of time the player has to complete the test. It has a `SerializeField`, so that `timeLimit` can be easily adjusted for each scene.

Updating the timer

The `Update()` method is responsible for continuously checking and updating the timer.

If the remaining time is less than or equal to zero and the timer is still running, it resets and stops the timer, then calls `SwitchPopup` to enable the `FailTimerContainer VisualElement`. This element is shown to the player, suggesting that they try again.

If the timer is still active and the remaining time is greater than zero, it decreases the `timeRemaining` variable by the value of `Time.deltaTime`. `Time.deltaTime` in Unity represents the time (in seconds) that has elapsed since the last frame was rendered. Then it calculates the number of minutes and seconds left.

- To calculate minutes, it divides `timeRemaining` by 60 and uses the `Mathf.FloorToInt` method to get the nearest whole number.
- It calculates seconds using the modulo operator (%) to get the remainder when dividing `timeRemaining` by 60, then applies `FloorToInt` method to obtain the whole number.

Finally, it checks if the seconds variable is less than ten. If so, it adds a leading zero before the seconds value. Otherwise, it updates the `timerLabel` with the current minutes and seconds.

```
1 private void Update()
2 {
3     // Disable timer and toggle the popup, if timer finished
4     if(timeRemaining <= 0 && isTimerRunning)
5     {
6         timeRemaining = 0;
7         timerLabel.text = "00:00";
8         StopTimer();
9         audioSource.Play();
10        PopupManager.SwitchPopup("FailTimerContainer", false, 0.07f,
11                                   true);
12    }
13    // Update the timer value
14    else if(isTimerRunning)
15    {
16        timeRemaining -= Time.deltaTime;
17
18        var minutes = Mathf.FloorToInt(timeRemaining / 60);
19        var seconds = Mathf.FloorToInt(timeRemaining % 60);
20        if(seconds < 10)
21        {
22            timerLabel.text = minutes + ":0" + seconds;
23        }else
24        {
25            timerLabel.text = minutes + ":" + seconds;
26        }
27    }
28 }
```

27 }

Listing 4.15: Method responsible for updating timer (source: author)

StartTimer is responsible for enabling the timer. It is called once the player clicks on the button to hide the introduction pop-up or when they click to restart the level. Once the player clicks, it switches to **isTimerRunning** and resets the time remaining.

StopTimer is responsible for stopping the timer. It changes the **isTimerRunning** variable to false to disable the timer.

OnTryAgainBtnClicked is responsible for activating the timer again in case the player didn't answer before the timer ended. It starts the timer by calling the **StartTimer** method and disables the **FailTimerContainer VisualElement** by calling **SwitchPopup**. This method is assigned to the button **btnRestartTimer**.

```
1 private void StartTimer()
2 {
3     isTimerRunning = true;
4     timeRemaining = timeLimit;
5 }
6
7 private void StopTimer()
8 {
9     isTimerRunning = false;
10 }
11
12 private void OnTryAgainBtnClicked()
13 {
14     StartTimer();
15     PopupManager.SwitchPopup("FailTimerContainer", true, 1f, false);
16 }
```

Listing 4.16: Methods responsible for starting, stopping, and restarting the timer (source: author)

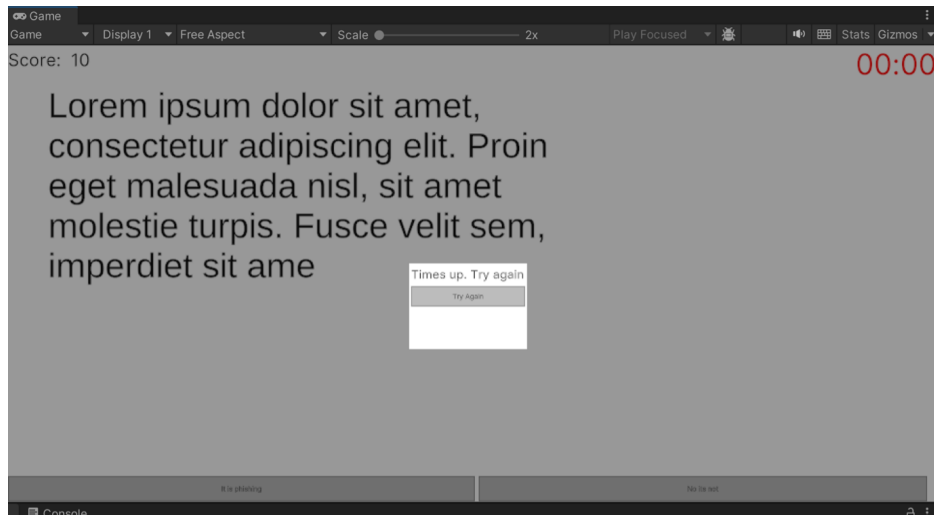


Figure 4.4: Example of *Guess test* level when the player runs out of time (source: author)

4.4.5 Score result

The script responsible for displaying the score result to the player in the final scene is called `ScoreResult`.

It contains only one method, `Start()`, which initialises a variable called `scoreResult`. This variable holds a reference to a `Label` element named `scoreResult`. After that, it updates the text value of this `Label` with the value of the score variable from `ScoreManager`.

```

1 private void Start()
2 {
3     var uiDocument = GameObject.FindObjectOfType<UIDocument>();
4     var root = uiDocument.root.GetComponent<VisualElement>();
5
6     // Update the scoreResult Label element with the score value
7     scoreResult = root.Q<Label>("scoreLabel");
8     scoreResult.text = "Final score: " + ScoreManager.Instance.score;
9 }

```

Listing 4.17: Method responsible for showing final score result to the player (source: author)

4.5 Hi-fi prototype

4.5.1 Visuals

Visuals play a crucial role in a game, as they help to establish the atmosphere and set the mood for the player. The game's story takes place in a futuristic distant future, so the backgrounds, UI, and fonts should align with this theme to create an immersive experience.

The author searched for assets on platforms such as itch.io and the Unity Asset Store. After checking various asset packs, the *Futuristic UI* pack created by author NYKNCK was selected. The pack was priced at \$16 and was fully paid by the author.

The author's page states that it is allowed to use assets in commercial projects and it is necessary to give credit to their name [21].

Here are the reasons why this asset pack was chosen:

- Consistency with the game's atmosphere - The science-fiction design enhances the futuristic setting, immersing the player in a high-tech world.
- Wide variety of assets – The pack provides a diverse selection of assets, allowing the creation of unique and visually distinct scenes.
- There is a wide variety of colours — The same assets are available in multiple colours, which helps differentiate their functions and improve usability.

Colour purpose

As mentioned above, the asset pack includes elements in four colours, forming the following colour palette.

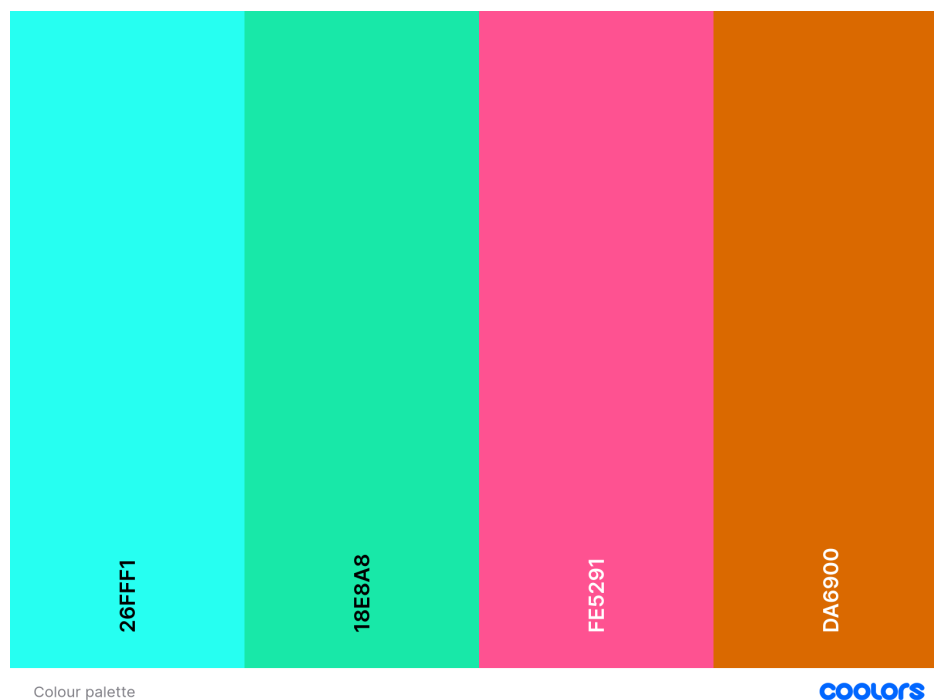


Figure 4.5: Illustration of a colour palette used to create visuals (source: author)

The author decided to assign a specific role to each colour to create a clear and intuitive UI, allowing the player to immediately understand the function of each element at a glance.

Here is the description of each colour and what purpose it fulfils in-game:

- Fluorescent Cyan (26FFF1) & Aquamarine (18E8A8) – used for the background in the scenes.
- French Rose (FE5291) – used for interactive UI elements: finish test button, it is a phishing button, it is not a phishing and buttons to navigate across the pop-up.
- Cocoa Brown (DA6900) – used for non-interactive but important UI elements, such as: score, timer, teacher’s name.

This structured approach improves visual clarity and ensures that players can easily differentiate between interactive and non-interactive elements.

4.5.2 Text font

Fonts play a crucial role in the game, especially when the player needs to read text frequently. The chosen fonts must be both visually fitting and easily readable to ensure a smooth experience.

Unity provides a selection of built-in fonts, but none matched the futuristic aesthetic of the game. After exploring different options, the author selected Valorax and Roboto-Bold as the primary fonts.

- Valorax – Chosen for its science-fiction appearance, enhancing immersion in the futuristic setting. This font is free for personal use [14].
- Roboto-Bold – Selected for phishing examples to distinguish them from the main game text. Since, according to the story, these examples originate from the distant past, a traditional email-style font was needed. Roboto-Bold, a built-in Unity font, was chosen because of its similarity to standard email typography. Roboto is an open-source font that is free for personal use [19].

Regular 400

Whereas recognition of the inherent dignity

Figure 4.6: Roboto-Bold font example [19]



Figure 4.7: Valorax font example [14]

4.5.3 Asset preparation

The creation of the background begins with the preparation of the assets. After downloading the asset pack, it cannot be used immediately. It is mandatory to prepare it first. The asset pack is initially downloaded as a ZIP file with different PNG images. Each image contains different sprites. The image needs to be sliced into sprites in order to use them. To achieve this, it is necessary to change the **Sprite mode** property to **Multiple**, so that Unity understands that the image holds multiple sprites.

Filter mode property is initially set to **Bilinear**, but it needs to be changed to **Point (no filter)**. **Filter mode** determines how textures are sampled when displayed at different resolutions or scales. The three options are:

- **Point (no filter)** - displays the texture without any smoothing or interpolation.
- **Bilinear** - smooths the texture by interpolating between neighbouring pixels.
- **Trilinear** - similar to bilinear, but also smooths across mipmap levels.

With the **Bilinear** option, the sprite state can be different from resolution to resolution, and it can be blurry. **Point (no filter)** ensures that each pixel is directly mapped to the screen pixels, preserving the sharp edges of the sprites [42].

In addition, the **Compression** property needs to be changed. It is initially set to **Normal quality**, but it needs to be changed to **None**. Texture compression reduces the file size of

your textures by simplifying the stored colour data. While this saves memory and improves performance, it can cause artefacts or distortions. **Compression** set to **None** helps keep the sprites' initial colour and avoids displaying unexpected artefacts.

After that, the image needs to be cut into assets, and the **Sprite Editor** is used to achieve this. There, we can choose how we want to slice the image, and it has three different properties:

- **Type** – sets the way the image is sliced. The default option is **Automatic**. There is no need to change it, Unity, because it is usually good at automatically slicing images.
- **Pivot** – the anchor point of a sprite. The default option is **Centre**. There is no need to change because it should match the pivot of the game object, which is set to the centre by default.
- **Method** - sets how to deal with existing sprites in the image. The default option is **Delete existing**. There is no need to change it because old sprites are not needed when the image is sliced again [38].

After the properties are set, the image can be sliced. Now, instead of images, we have sprites for each UI element.

To prepare the assets, the author was mostly using Unity's official documentation and the Youtube video called "How to Import a 2D Character Sprite Sheet and Use in a GameObject in Unity (2021)" from author Chris' Tutorials [35].

4.5.4 Background creation

To create a background in Unity, first we need to create a GO with a **Sprite Renderer** component. The **Sprite Renderer** has a property called **Sprite**. This property is used to assign a sprite to a GO. To assign an asset, we must drag and drop the asset onto the **Sprite** property.

To organise all the GOs associated with the background, they were placed inside an empty object called **BG**. This approach makes the GOs structure inside the scene look cleaner and easier to read. In addition, **BG** was used as a prefab because most of the scenes have the same UI.

Introduction to the story, Score result

The background for the introduction and final level is the same, and it looks like this.



Figure 4.8: Introduction to story level visuals representation (source: author)



Figure 4.9: Score result level visual representation (source: author)

Pop-up window

The *Hint test* and *Guess test* levels start with an open introduction pop-up, so the author decided to create a background for it first. The background of the pop-up window is stored in an empty GO called `PopupBG`. This GO was also made as a prefab because it is present in almost every scene.

The asset used to visualise the teacher in the pop-up was taken from an asset pack called *Free Pixel Art Cyberpunk Character Portraits Pack* by The Outlander. The creator of the asset pack states that the assets can be used in any project and do not require any form of credit [32].



Figure 4.10: Pop-up visual representation (source: author)

Hint test and Guest test

The `Hint test` and the `Guess test` visuals are almost the same. The only difference is that the `Hint test` contains information about the total number of suspicious elements in the level. In contrast, the `Guess test` shows either an empty timer slot or how much time is left to finish the level. The final result looks like this.

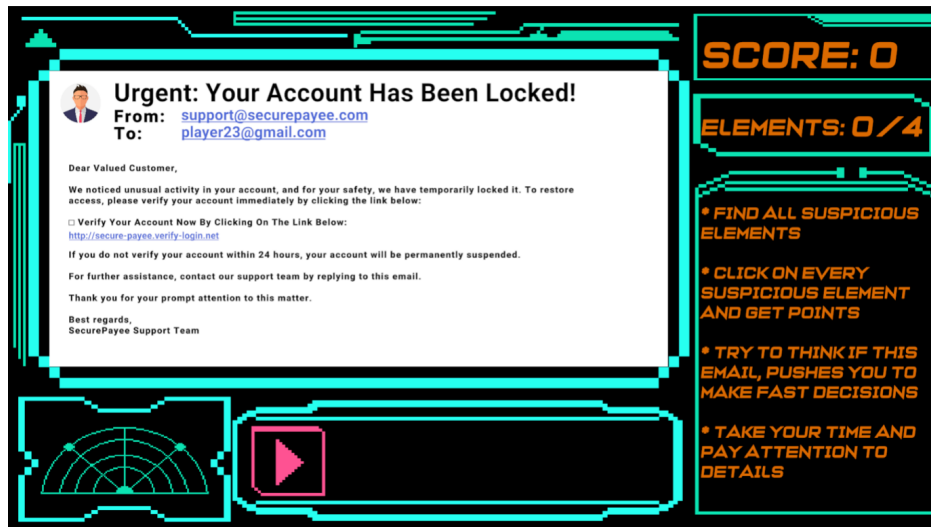


Figure 4.11: Hint test visuals representation (source: author)



Figure 4.12: Guess test visuals representation (source: author)

4.5.5 Teacher lines and phishing examples

As was mentioned earlier, the game has four main scenes. Each scene can have a different type of text, such as:

- introduction,
- explanation,
- success,
- failure.

Each scene also has a unique example of phishing. All text inside the pop-up windows is the teacher's lines. To create lines for the teacher, the author used artificial intelligence, specifically ChatGPT. It was used because the main goal of this thesis is not to create an original and entertaining story. ChatGPT was also used to generate phishing examples. For the first phishing example, a phishing email with two suspicious elements was required.

For the **Hint test 1**, it was required to generate lines of the introduction of the studied subject by the player, introduce phishing, give an example, explain how to finish the level and what to pay attention to. For the phishing example, it was required to generate a phishing example with a generated greeting, a suspicious link, and an email. In addition, it was required to generate an explanation for each suspicious spot, which was used as explanation slides.

For the **Hint test 2**, it was required to generate an email with suspicious elements from the previous level and add urgency to the email. For the introduction, it was required to generate an explanation of urgency-based phishing emails.

For the **Guess test 1**, it was required to generate an email that looks like a phishing email but is, in reality, a valid email. For the introduction, it generated an explanation of how new levels work.

For the **Guess test 2**, a phishing email using suspicious elements from previous scenes was required. An explanation of how the timer works was generated for the introduction.

5. Testing

After finishing the Hi-fi prototype, it is necessary to show the game to others in order to gather valuable feedback. This feedback helps identify improvement areas, ensuring that the game is polished before its release.

5.1 Sharing the game

To test the game, it was necessary to share it with the testers. There are two ways to do this:

- *Standalone executable* — is a build of the game created for a specific platform. The advantage of this method is that the player does not need to install additional software. However, the downside is that a separate build must be created for each platform, and executable files can be large in size, requiring compression.
- *WebGL build* – is a build that allows one to share the game via a website or hosting platform. The advantage of this method is that players only need a web browser to launch the game, eliminating the need for platform-specific builds. In addition, it provides a quick and easy way to share the game, especially when testing prototypes. The main drawback is that performance may be lower compared to standalone builds.

For this project, WebGL build was chosen due to the author's previous experience with it, the small size of the project, and the challenges of sharing a large standalone executable with low internet speed.

After creating the build, it needed to be deployed on a hosting platform so that testers could access it. Itch.io was selected because the author had prior experience with it and the tutorial also demonstrated the deployment process on this platform.

To create and share the WebGL build, the author was following the Youtube video named "How To Upload your Unity Game to Itch.io using WebGL" from author Indie Game Academy [1].

5.2 Feedback

This chapter was divided into two parts. In the first part, the author collected and analysed the opinions of the testers about the visuals and mechanics of the game. The second part describes the analysis of how well the game accomplishes its main goal: increasing awareness about phishing.

5.2.1 Gameplay and visuals

After publishing the game, it was shared with three testers. During their playthrough, the author observed their interactions with the game while they shared their thoughts, highlighting aspects that they liked and areas that they found problematic. This approach allowed the author to identify players' difficulties and make necessary improvements to enhance clarity and overall enjoyment.

After completing their playthroughs, the testers provided a summary of their final opinions. Below is their feedback, categorised into aspects they liked, aspects they disliked, and their suggestions for improvement.

Tester 1

Liked

- The game's overall atmosphere and UI design.

Issues identified

- Too much text at the beginning of the first level - The tester found the introduction too long, especially in the first level, which made it less engaging.
- Lack of goal recap within the level - Due to the lengthy introduction, the tester skipped the text and, as a result, did not understand the main objective. He suggested adding an element within the scene that concisely recaps the level's goal.

Suggestions for improvement

- Reduce the amount of text in the introduction.
- Add an element summarising the level's goal.
- Consider changing the font.

Tester 2

Liked

- The core game concept.

Issues identified

- Font readability issues - Similar to the first tester, he found the font difficult to read.
- Lack of feedback when clicking on suspicious elements - The tester was unsure whether their actions were registered. They suggested adding sound feedback to indicate a successful action.

- Lack of penalties for incorrect actions - The game does not penalise wrong choices, making it feel less challenging. The tester suggested reducing points if the timer runs out.
- No indication of previously clicked words - The tester found it difficult to remember which words had already been identified as suspicious and suggested highlighting them.

Suggestions for improvement

- Implement sound feedback when interacting with suspicious elements.
- Introduce score penalties for running out of time or making incorrect choices.
- Highlight previously clicked words in **Hint test** levels.

Tester 3**Liked**

- The game's mechanics and educational value.

Issues identified

- Lack of sound feedback - Similar to the second tester, they suggested adding sound effects to improve interaction feedback.
- No background music - The absence of music made the game feel less immersive.
- Lack of penalties for mistakes - He agreed with the second tester that the game should have more penalties for incorrect choices.
- UI transparency bugs - During the explanation phase, the score label and the **Finish Test** button were not transparent, leading to visual clutter.

Suggestions for improvement

- Implement sound effects for user actions.
- Add background music to enhance immersion.
- Introduce point deductions for incorrect choices.
- Fix transparency issues in the UI.

5.2.2 Main goal accomplishment

To test the game's effectiveness as an educational tool, the author created a questionnaire, which testers completed once before starting the game and again after finishing it. This approach allows to compare the knowledge of the testers about phishing before and after playing the game and see if it increases their understanding of the problem.

The questionnaire consists of 8 questions. The first five are single-choice questions, which test the testers' knowledge about phishing. The other three questions are examples of emails, and

the tester must choose whether each is a phishing attempt. All questions relate to the topics the player covers while playing the game. The questions and email examples were generated with the help of artificial intelligence, specifically ChatGPT. Here is the list of questions:

1. What is phishing?
2. What is the main goal of a phishing attack?
3. Which of the following is a common sign of a phishing email?
4. What types of personal information do phishing attacks commonly target?
5. How can you tell if an email is from a trusted sender?

To see the questionnaire, you can check Google form. Here is a chart comparing the results of three testers before and after playing the game.

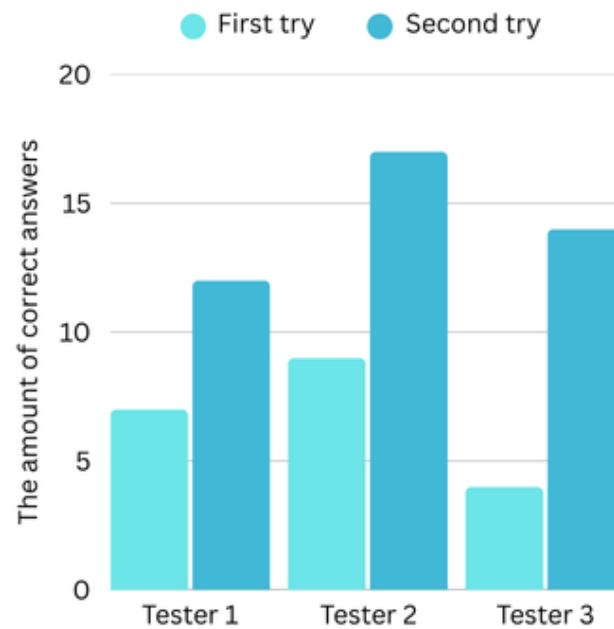


Figure 5.1: Bar graph representing testers' results in the questionnaire (source: author)

The average success rate among all testers during the first round was 42.11%. It increased by 33.33% during the second round, reaching 75.44%. The increase in the success rate in the second round is a sign that the game succeeds in its main goal and can be used as an educational tool.

6. Pre-launch

After collecting and organising the feedback from the testers, the author needed to identify key areas for improvement and determine the necessary mechanics to enhance the game cycle. These mechanics should refine the gameplay experience, making it more engaging, intuitive, and enjoyable for players.

6.1 Feedback analysis

After analysing all the feedback, the author compiled a list of mechanics that needed improvement or updates. The testers commonly mentioned the following functionality, and the author decided that adding it would increase engagement and make the gameplay smoother.

Here are these mechanics and an explanation of why the author decided to add them:

- Change font - This is the most common issue mentioned by the testers. The game has a lot of text to read, so this needs to be addressed first.
- Show the number of items the user still needs to click on - This wasn't mentioned by testers but is still an important mechanic. After observing how the testers played the game, the author noticed that they often spent too much time on **Hint test** levels because they didn't know if they had spotted all the suspicious elements. With this mechanic, players will have a clear understanding of their progress.
- Add a recap of the level's goal - This will help players recall the main goal of the mission in case they forget it.
- Highlight elements that have already been clicked on - Similar to the previous mechanic, this helps players track which suspicious elements they have already spotted, making their progress clearer.
- Modify pop-up navigation - This mechanic will prevent players from leaving the pop-up until they have read all slides. It ensures that they see all important information about the level and don't miss anything.
- Add sound feedback - This mechanic will add sound effects when players click on buttons in the game. It will be especially helpful in the **Hint test** when players click on suspicious elements, providing clear feedback for their actions.
- Add background music - This will increase immersion and make the game feel more vibrant.
- Penalties for mistakes - This will encourage the player to think carefully before making a decision and to promote better choices.

6.2 Change font

The previous font used as the main font for dialogues had two problems: it did not contain numbers and was hard to read. After searching for a different font that would solve these issues while still matching the game's atmosphere, the author decided to use Venite Adoremus. This is what it looks like in the game. The author of this font states that the font is free only for non-profit projects [12].

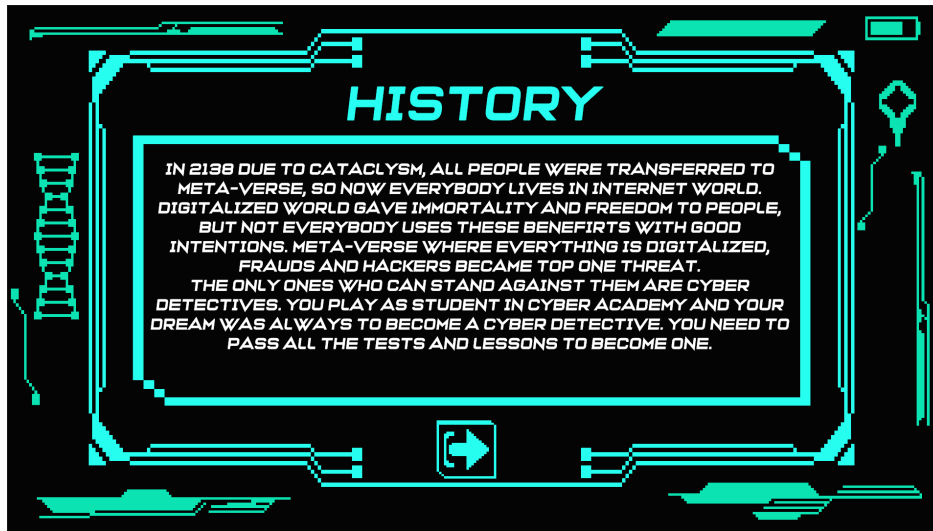


Figure 6.1: Venite Adoremus font representation (source: author)

6.3 Show the number of items the player still needs to click on

First, a new UI element needed to be added to show the player the total number of suspicious elements and how many they had already clicked on. A new `VisualElement` named `SuspiciousElementCounter` was added to each *Hint test* `UIDocument`. This `VisualElement` was placed inside another `VisualElement` called `SceneInfoContainer`, so that its opacity changes when the pop-up becomes visible. It contains three elements:

- `mainLabel` – a `Label` element explaining to the player what this section represents. Displays the following text "Suspicious elements".
- `elementsClicked` – a `Label` showing the number of suspicious elements the player has already clicked on.
- `elementsTotal` – a `Label` displaying the total number of suspicious elements hidden in the level.

At the start of the level, the game needs to count the total number of suspicious elements and update the `elementsTotal` variable text value. After clicking on a suspicious element, the `elementsClicked` text value updates accordingly.

All changes were made inside the `HintObjectManager` script. At the beginning, the variables

`suspiciousElementClicked` and `suspiciousElementsTotal` were created to reference the respective labels from the `UIDocument`. The variable `suspiciousElementsCounter` was also created to track the number of elements the player had already clicked on.

The variables `suspiciousElementsTotal`, `suspiciousElementClicked`, and `suspiciousElementsCounter` were initialized in the `Start` method. The default value of `suspiciousElementsCounter` is set to zero, and the `suspiciousElementsTotal` text value is updated to match the length of the `hintObjects` list.

```
1 suspiciousElementsCounter = 0;
2 suspiciousElementClicked = root.Q<Label>("elementsClicked");
3 suspiciousElementsTotal = root.Q<Label>("elementsTotal");
4 suspiciousElementsTotal.text = "/" + hintObjects.Count.ToString();
```

Listing 6.1: Code responsible for initialising variables used for suspicious elements counter (source: author)

In the `OnClickHintObject` method, the `suspiciousElementsCounter` value increases by one, and the `suspiciousElementClicked` text value is updated to reflect the number of elements the player has clicked on.

```
1 suspiciousElementsCounter++;
2 suspiciousElementClicked.text = suspiciousElementsCounter.ToString();
```

Listing 6.2: Code responsible for updating suspicious elements counter status (source: author)

6.4 Add a recap of the level's goal

The UI needed to be updated to add a recap of the level. Previously, the right side of the scene was used only as a background and had no functionality. This background was removed to create space for the recap UI element.

In each `Hint test` `UIDocument`, a new `VisualElement` named `RecapGoal` was added. It contains a single `Label` element displaying key points the player must remember while playing the level.

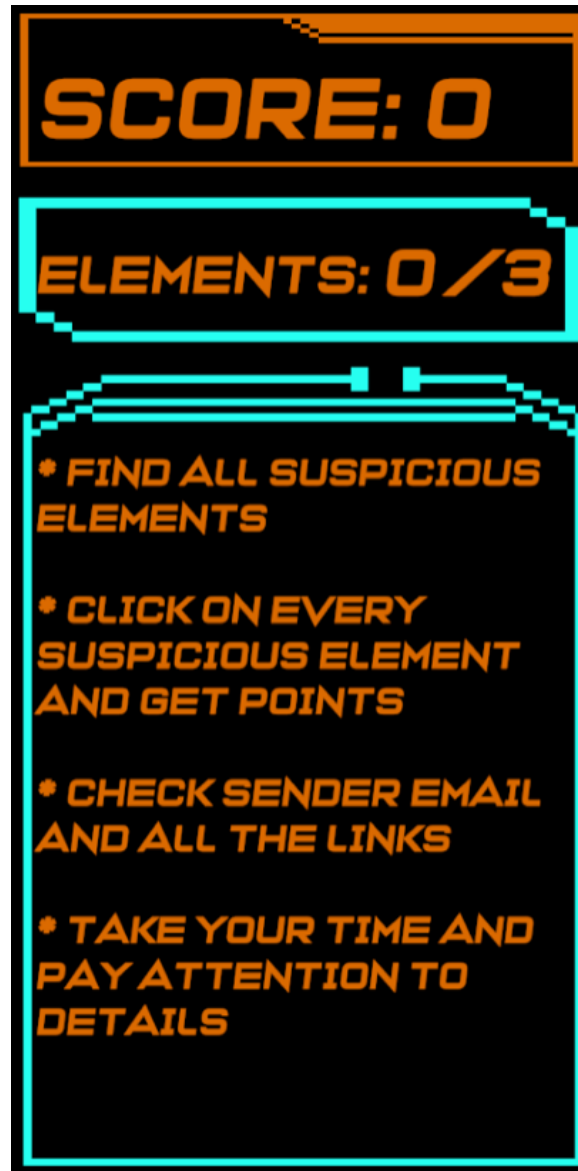


Figure 6.2: Level's goal recap visual representation (source: author)

6.5 Highlight the element that was already clicked on

After clicking on a suspicious element, it should be highlighted in red. Previously, the `OnClickHintObject` method removed clicked items, preventing the player from clicking on them again. Instead of deleting the elements, they should remain visible and be highlighted in red.

To achieve this, a `Sprite Renderer` component with a red colour was added to each suspicious element. The `Sprite Renderer` was disabled and should only be enabled after the player clicks on the element.

All code modifications were made within the `HintObjectManager` script. Inside the `OnClickHintObject` method, instead of deleting the suspicious element, the script finds the reference to the `Box Collider 2D` component of the clicked element and disables it. With `Box Collider 2D` disabled, clicks on the object will no longer be registered, preventing the player from clicking on it again to earn extra points. To highlight the object, the script also finds the reference to the `Sprite Renderer` of the current suspicious element and enables it.

```
1 // Disable the box collider and enable the sprite renderer
2 BoxCollider2D currentH0boxCollider = currentHintObject.GetComponent<
  BoxCollider2D>();
3 currentH0boxCollider.enabled = false;
4 SpriteRenderer currentH0spriteRenderer = currentHintObject.
  GetComponent<SpriteRenderer>();
5 currentH0spriteRenderer.enabled = true;
```

Listing 6.3: Code responsible for updating the `Sprite Renderer` of the respective suspicious element (source: author)

6.6 Modify pop-up navigation

The button to close the introduction pop-up and the button to proceed to the next level should be unclickable until the player reaches the last slide. When the user reaches the last slide, it changes its opacity and becomes clickable. All code modifications were performed inside the `SlideManager` script. In the `Start` method, a variable called `btnHideIntro` is initialised. The button is disabled, so the player cannot click on it.

```
1 btnHideIntro = root.Q<Button>("btnHideIntro");
2 btnHideIntro.SetEnabled(false);
```

Listing 6.4: Code responsible for disabling the `btnHideIntro` (source: author)

After that, the `Update` method checks if the player reached the last slide. If he did, then the opacity of the button will be changed, and it will be enabled.

```
1  // Disable btnPrevSlide and enable btnHideIntro if the current slide
   is the last one
2  if (currentSlideIndex == slidesArray.Length - 1)
3  {
4      btnNextSlide.style.opacity = .07f;
5      btnNextSlide.SetEnabled(false);
6
7      btnHideIntro.style.opacity = 1f;
8      btnHideIntro.SetEnabled(true);
9  }
```

Listing 6.5: Code responsible for loading the next scene (source: author)

6.7 Penalties for mistakes

Two out of three testers highlighted the lack of penalties in the game. After analysing, the author decided to add penalties only if the timer ends before the player makes this choice. **Guess test** already had a penalty for wrong answers, and adding penalties to **Hint test** would be a bad choice because it could overwhelm the player at the beginning of the game.

The **Update** method inside the **TimerManager** script was modified to add this functionality. If the time left is below zero and the timer is still active, it will decrease the player's score by five points.

```
1  if(timeRemaining <= 0 && isTimerRunning)
2  {
3      timeRemaining = 0;
4      timerLabel.text = "00:00";
5      StopTimer();
6      audioSource.Play();
7
8      PopupManager.SwitchPopup("FailTimerContainer", false, 0.07f, true)
9      ;
10
11     ScoreManager.Instance.AddScore(-5);
12 }
```

Listing 6.6: Code responsible for decreasing the score after the player runs out of time (source: author)

6.8 Sound feedback

All buttons should play a specific sound after the player clicks on them. There are three possible sounds that buttons can play:

- *Click sound* – plays when the player clicks on one of the pop-up buttons or on a suspicious element.
- *Success sound* – plays when the player clicks the button to finish the test in `Hint test` or selects the correct option in `Guess test`.
- *Failure sound* – plays when the player fails to finish the test on time or selects the wrong option in the `Guess test`.

6.8.1 Finding the sounds

First, it is necessary to find source files that will be played after a certain action. The author checked the Unity Asset Store and decided to go with the *Casual Game SFX Pack* because it is free and provides a wide variety of different sounds. It is under the Unity Extension Asset licence, which allows free usage for commercial projects [37].

6.8.2 Click sound

The script responsible for playing the Click sound is called `ButtonSoundManager`. In the first scene, at least one button always needs to play a Click sound, so this script was designed as a singleton.

First, to manipulate the sound inside the script, it is necessary to attach it to a game object along with the script. In the `IntroToStory` scene, an empty GO called `ButtonSoundManager` was created. To this GO was assigned the `ButtonSoundManager` script and an `Audio Source` component.

The `Audio Source` is responsible for playing sounds in the scene. It has different properties, such as `Audio Clip`, `Loop`, `Mute`, `Play on Awake`.

`Play on Awake` is enabled by default, but since the sound should only play after a button click, it was disabled. The `Audio Clip` property holds a reference to the sound file that will be played. The author selected a sound from the chosen pack that best suited the scene and assigned it to the `Audio Clip` property.

`RegisterButtons` is a public method responsible for finding all buttons in the current scene and making them play a sound when clicked. The `audioSource` variable holds a reference to the `Audio Source` component assigned to the `ButtonSoundManager` GO. It has a built-in `Play` method that plays the assigned `Audio Clip`. The `RegisterButtons` method is called inside the `SceneManager` script because every scene has this script.

```
1 public class ButtonSoundManager : MonoBehaviour
2 {
3     // Singleton instance of ButtonSoundManager, allowing global
4     // access to its methods and properties
5     public static ButtonSoundManager Instance { get; private set; }
```

```
5
6      // Holds the reference to the audio source
7      private AudioSource audioSource;
8
9      private void Awake()
10     {
11         // Check if other instance exists
12         if (Instance != null && Instance != this) {
13             Destroy(gameObject);
14             return;
15         }
16
17         // Set this as the Singleton instance
18         Instance = this;
19
20         // Prevent this object from being destroyed when loading a new
21         scene
22         DontDestroyOnLoad(gameObject);
23
24         audioSource = gameObject.GetComponent<AudioSource>();
25     }
26
27     public void RegisterButtons(UIDocument uiDocument) {
28         // Initialize all the buttons in the current scene
29         var root = uiDocument.root\texttt{VisualElement};
30         var buttons = root.Query<Button>().ToList();
31
32         // Assign a method to play the sound of the click event
33         foreach (var btn in buttons)
34         {
35             btn.clicked += () => audioSource.Play();
36         }
37     }
38 }
```

Listing 6.7: Code responsible for managing sounds for buttons (source: author)

6.8.3 Success sound and Failure sound

The respective GOs needed to have an `Audio Source` component to play both sounds. In the *Hint test*, an `Audio Source` was added to the GO `HintObjectManager`. The same was done for `GuessTestManager` and `TimerManager` in *Guess test*. Each `Audio Source` was assigned the corresponding sound:

- Success sound to `HintObjectManager`.
- Failure sound to `TimerManager`.

To make `HintObjectManager` play the sound, a method call was added to `OnClickFinishTest`. Similarly, in `TimerManager`, `audioSource.Play()` was added in the `Update` method when the timer ends.

With `GuessTestManager`, the situation was different because it needed to play a different sound depending on the player's answer. To achieve this functionality, two variables were created:

- `audioClipFail` - for the failure sound.
- `audioClipWin` - for the success sound.

Both variables were marked with the `SerializeField` attribute, allowing audio files to be assigned in the Unity.

The `ShowMessage` method was updated to include a new parameter called `audioClip`. Inside `ShowMessage`, the `Audio Clip` is updated based on the value passed when the method is called. `ShowMessage` is triggered inside the methods `OnClickIsNotPhishing` and `OnClickIsPhishing`. Depending on the value of the `isPhishing` variable, it passes either `audioClipWin` or `audioClipFail` as a parameter.

```
1 private void ShowMessage(string msg, AudioClip audioClip, int points)
2 {
3     // Update the score value
4     ScoreManager.Instance.AddScore(points);
5     // Update the text value of the first slide
6     Label firstSlide = slidesExplanation[0];
7     firstSlide.text = msg + firstSlide.text;
8
9     // Toggle the popup status
10    var PopUpManager = FindObjectOfType<PopupManager>();
11    PopUpManager.SwitchPopup("PopUpExplanationContainer", false, 0.07f
12        , true);
13
14    // Play the sound
15    audioSource.clip = audioClip;
16    audioSource.Play();
17 }
```

Listing 6.8: Code responsible for playing respective sounds after the player finishes *Guess test* level (source: author)

6.8.4 Background music

First of all, it is necessary to find background music that suits the game's atmosphere. The audio pack used for button sounds did not contain any appropriate tracks, so the author searched for one in the Unity Asset Store. The *Sci-Fi and Cyberpunk Music Pack* was chosen because it was free and included long audio tracks that matched the atmosphere of the game.

The same as the asset pack used for button sounds, this one is under Unity Extension Asset licence [36].

To add background music to the game, a script called **BgMusicManager** was created. In the **IntroToStory** scene, an empty GO named **BgMusicManager** was created. An **Audio Source** component was then attached to this GO.

The **Audio Clip** was assigned an audio file called **Ascension** from the selected audio pack. Furthermore, the **Loop** and **Play on Awake** properties were enabled because the background music had to start playing immediately and repeat once the track was finished.

The **BgMusicManager** script contains only one method, **Awake**. Inside, the Unity built-in method **DontDestroyOnLoad** is called to ensure that the background music continues to play when switching to the next scene.

```
1 public class BgMusicManager : MonoBehaviour
2 {
3     void Awake()
4     {
5         //Keep the music playing after transitioning to the next scene
6         DontDestroyOnLoad(gameObject);
7     }
8 }
```

Listing 6.9: Code responsible for playing background music (source: author)

Conclusion

The main goal of this thesis was to create a game that would increase awareness of the phishing problem. The practical part focused on researching SGs, analysing different games, and creating the final concept. The practical part also focused on development, testing, and analysing feedback to improve the game.

In the beginning of the theoretical part, research was conducted on SG to better understand the topic using available documents. Subsequently, the initial concept was created, which was then used to identify similar SGs and analyse them using comparison, observation, and document analysis methods. Based on the results of this analysis, the final concept was created. The final concept includes a detailed description of the game's storyline, visuals, functionalities, and levels. A list of features was compiled and prioritised using the MoSCoW method, distinguishing between those to be implemented during the thesis and those suitable for future development.

The practical part provides a detailed account of the development process. It begins with an overview of the tools selected for development: the Unity game engine and the C# programming language. The process started with the creation of a Lo-fi prototype, which outlines the implementation of the game engine's core functionality, including the development of the UI, game mechanics, and basic visuals. This was followed by the Hi-fi prototype, which involved creating content such as teacher dialogue and phishing emails. During this phase, assets were prepared and final visuals were implemented. The entire process is documented with code examples and in-game screenshots.

The next stage was testing. It began with a description of how the game was distributed to testers, followed by a presentation of the testing results. First, the visual and functional quality of the game was evaluated through user interviews. Each tester provided feedback on what they liked, what they disliked, and suggestions for improvement. Second, the educational effectiveness of the game was assessed using a questionnaire. This questionnaire included five phishing-related questions and three example emails, which participants were asked to classify as either phishing or legitimate. Each participant completed the questionnaire both before and after playing the game. The results, shown in a bar chart, indicated a 33.33% increase in correct responses during second try, demonstrating the game's potential as an effective educational tool.

In the end, the feedback of all the testers on the quality of the game was collected and analysed. Based on this, a list of functionalities needed to improve the player experience and increase engagement was compiled. Specifically, the game was enhanced with background music, sound feedback when a player clicks a button, a total counter of suspicious elements, and a recap of each level's main goal. In addition, several mechanics were modified, including pop-up navigation, highlighting suspicious elements after clicking, font updates, and penalties for mistakes.

The MoSCoW method was used to outline a future development plan for the game. This plan includes adding more levels and difficulty modes, releasing the game on iOS and Android platforms, implementing voice-overs for the characters, and introducing accessibility support. Additionally, the game could broaden its educational scope, similar to what *Interland* does — for example, by introducing new levels that address other cybersecurity threats.

Although the game was evaluated as an educational tool with promising results, the number of testers was limited. As mentioned in the theoretical section, there is a general lack of empirical evidence on the effectiveness of SGs in real-world scenarios. Future work could focus on more extensive research into the educational value of SGs, using the game developed in this thesis as a case study. Such research could inform further improvements and support the game's implementation across a broader range of educational institutions.

References

- [1] Indie Game Academy. *How To Upload your Unity Game to Itch.io using WebGL*. Video. Aug. 2024. URL: <https://www.youtube.com/watch?v=-oRgI54fcbI>.
- [2] Ernest Adams and Joris Dormans. *Game Mechanics: Advanced Game Design*. English. [S.l.]: New Riders, 2012.
- [3] Ali Amini. *MIT License (Markdown version)*. n.d. URL: <https://www.mit.edu/~amini/LICENSE.md>.
- [4] G. Aparna et al. ‘Phishing Simulations’. In: *International Journal of Novel Research in Engineering & Pharmaceutical Sciences* 3.2 (2024), p. 866. URL: https://www.researchgate.net/publication/390024412_PHISHING_SIMULATIONS.
- [5] APWG. *Phishing Activity Trends Report*. Mar. 2025. URL: https://docs.apwg.org/reports/apwg_trends_report_q4_2024.pdf.
- [6] BreakAway Games. *Free Dive*. n.d. URL: <https://www.breakawaygames.com/case-studies/free-dive/>.
- [7] Johannes Breuer and Gary Bente. ‘Why So Serious? On the Relation of Serious Games and Learning’. In: *Journal for Computer Game Culture* 4.1 (2010), pp. 7–24. URL: <https://hal.science/hal-00692052v1/document>.
- [8] Karen Brush. *MoSCoW Method*. TechTarget. Mar. 2023. URL: <https://www.techtarget.com/searchsoftwarequality/definition/MoSCoW-method>.
- [9] catnikma. *Unknown person silhouette with glasses. profile picture, silhouette profile. man avatar profile. male profile icons*. n.d. URL: <https://depositphotos.com/vector/unknown-person-silhouette-glasses-profile-picture-silhouette-profile-man-avatar-179308454.html>.
- [10] Common Sense Media. *Hungry Red Planet Review*. n.d. URL: <https://www.common sense media.org/game-reviews/hungry-red-planet>.
- [11] Amanda Conway. *Game Design Document*. eCampus Ontario Pressbooks. 2021. URL: <https://ecampusontario.pressbooks.pub/gamedesigndevelopmenttextbook/chapter/132/>.
- [12] DaFont. *Venite Adoremus Font*. n.d. URL: <https://www.dafont.com/venite-adoremus.font>.
- [13] Laura Ermi and Frans Mäyrä. ‘Fundamental Components of the Gameplay Experience: Analysing Immersion’. In: *Proceedings of DiGRA 2005 Conference: Changing Views: Worlds in Play*. Tampere: DiGRA, 2005. URL: <https://dl.digra.org/index.php/dl/article/view/119>.
- [14] Font Meme. *Valorax Font*. n.d. URL: <https://fontmeme.com/fonts/valorax-font/#fontinfo>.
- [15] GeeksforGeeks. *Singleton Method Design Pattern*. Unity Documentation. 3rd Jan. 2025. URL: <https://www.geeksforgeeks.org/singleton-design-pattern/>.

- [16] GitHub. *About GitHub and Git*. n.d. URL: <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>.
- [17] Google. *Be Internet Awesome*. n.d.-a. URL: https://beinternetawesome.withgoogle.com/en_us.
- [18] Google. *Interland*. n.d.-b. URL: https://beinternetawesome.withgoogle.com/en_us/interland.
- [19] Google Fonts. *Roboto*. n.d. URL: <https://fonts.google.com/specimen/Roboto/license>.
- [20] Kai Huotari and Juho Hamari. 'Defining Gamification: A Service Marketing Perspective'. In: *Proceedings of the 16th International Academic MindTrek Conference*. 2012, pp. 17–22. URL: https://www.researchgate.net/publication/259841647_Defining_Gamification_-_A_Service_Marketing_Perspective.
- [21] Itch.io. *NYKNCK Community Profile*. n.d. URL: <https://nyknck.itch.io>.
- [22] Nathan King. *Serious Games Guide: Everything You Need to Know in 2024*. Chaos Theory Games. 19th Sept. 2024. URL: <https://www.chaostheorygames.com/blog/serious-games-guide-everything-you-need-to-know-in-2024>.
- [23] Nathan King. *Serious Games vs. Gamification: What's the Difference?* Chaos Theory Games. 31st May 2021. URL: <https://www.chaostheorygames.com/blog/serious-games-vs-gamification-difference-benefits>.
- [24] MIT Technology Licensing Office. *Exploring the MIT Open Source License: A Comprehensive Guide*. n.d. URL: <https://tlo.mit.edu/understand-ip/exploring-mit-open-source-license-comprehensive-guide>.
- [25] My Abandonware. *Hungry Red Planet*. n.d. URL: <https://www.myabandonware.com/game/hungry-red-planet-m42>.
- [26] Rich Newman. *Cinematic Game Secrets for Creative Directors and Producers: Inspired Techniques from Industry Legends*. Accessed via ProQuest Ebook Central. CRC Press LLC, 2009. URL: <https://ebookcentral.proquest.com/lib/vsep/detail.action?docID=4689318>.
- [27] Pavlo Prots. *The 7 Stages of the Game Development Process*. Accessed on 2025-05-02. Stepico. Dec. 2023. URL: <https://stepico.com/blog/game-development-stages/>.
- [28] A. Shuhalii. *What Is the Difference Between Low and High Fidelity Prototypes?* Medium. 16th Oct. 2020. URL: <https://medium.com/design-bootcamp/what-is-the-difference-between-low-and-high-fidelity-prototypes-b1f3612f85f7>.
- [29] SteamDB. *Frequently Asked Questions*. n.d. URL: <https://steamdb.info/faq/#why-are-we-doing-this>.
- [30] SteamDB. *What Are Games Built With and What Technologies Do They Use?* 2025. URL: <https://steamdb.info/tech/>.

-
- [31] Tarja Susi, Mikael Johannesson and Per Backlund. *Serious Games – An Overview*. Technical Report HS-IKI-TR-07-001. School of Humanities and Informatics, University of Skövde, 2015. URL: <https://www.diva-portal.org/smash/get/diva2:2416/FULLTEXT01.pdf>.
- [32] The Outlander. *Free Pixel Art Cyberpunk Character Portraits Pack*. n.d. URL: <https://the-outlander.itch.io/free-pixel-art-cyberpunk-character-portraits-pack>.
- [33] Alan Thorn. *Game Engine Design and Implementation*. Jones & Bartlett Learning, 2010. URL: <https://books.google.cz/books?id=cFnPvWK9-akC>.
- [34] Roberto Trinchero and Giovanni Sala. ‘Chess Training and Mathematical Problem-Solving: The Role of Teaching Heuristics in Transfer of Learning’. In: *EURASIA Journal of Mathematics, Science and Technology Education* 12.3 (2016), pp. 655–668. URL: <https://www.ejmste.com/download/chess-training-and-mathematical-problem-solving-the-role-of-teaching-heuristics-in-transfer-of-4483.pdf>.
- [35] Chris’ Tutorials. *How to Import a 2D Character Sprite Sheet and Use in a GameObject in Unity (2021)*. Video. May 2021. URL: <https://www.youtube.com/watch?v=FXXc0hTWIMs&t=301s>.
- [36] Unity Asset Store. *Free – Sci-Fi and Cyberpunk Music Pack*. n.d. URL: <https://assetstore.unity.com/packages/audio/ambient/sci-fi/free-sci-fi-and-cyberpunk-music-pack-264590>.
- [37] Unity Asset Store. *Free Casual Game SFX Pack*. n.d. URL: <https://assetstore.unity.com/packages/audio/sound-fx/free-casual-game-sfx-pack-54116?srsId=AfmB0oqT7Ljc7crpNmz7Ecuo7KnsRmEYJfmahUjEoehcPdZ5az1RKCR2>.
- [38] Unity Technologies. *Automatic Slicing*. n.d.-o. Unity Documentation. URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/sprite/sprite-editor/automatic-slicing.html>.
- [39] Unity Technologies. *BoxCollider2D*. n.d.-l. Unity Documentation. URL: <https://docs.unity3d.com/ScriptReference/BoxCollider2D.html>.
- [40] Unity Technologies. *Comparison of UI Systems in Unity*. n.d.-d. Unity Documentation. URL: <https://docs.unity3d.com/Manual/UI-system-compare.html>.
- [41] Unity Technologies. *Creating Scenes*. n.d.-a. Unity Documentation. URL: <https://docs.unity3d.com/Manual/CreatingScenes.html>.
- [42] Unity Technologies. *FilterMode*. n.d.-n. Unity Documentation. URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/FilterMode.html>.
- [43] Unity Technologies. *GameObject*. n.d.-b. Unity Documentation. URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/GameObject.html>.
- [44] Unity Technologies. *Introduction to Components*. n.d.-c. Unity Documentation. URL: <https://docs.unity3d.com/Manual/Components.html>.
-

- [45] Unity Technologies. *Mathf*. n.d.-p. Unity Documentation. URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Mathf.html>.
- [46] Unity Technologies. *MonoBehaviour*. n.d.-f. Unity Documentation. URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/MonoBehaviour.html>.
- [47] Unity Technologies. *MonoBehaviour.OnMouseDown()*. n.d.-m. Unity Documentation. URL: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.OnMouseDown.html>.
- [48] Unity Technologies. *MonoBehaviour.Start()*. n.d.-h. Unity Documentation. URL: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.Start.html>.
- [49] Unity Technologies. *MonoBehaviour.Update()*. n.d.-i. Unity Documentation. URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/MonoBehaviour.Update.html>.
- [50] Unity Technologies. *Prefabs*. n.d.-e. Unity Documentation. URL: <https://docs.unity3d.com/Manual/Prefabs.html>.
- [51] Unity Technologies. *SceneManager*. n.d.-j. Unity Documentation. URL: <https://docs.unity3d.com/ScriptReference/SceneManagement.SceneManager.html>.
- [52] Unity Technologies. *SerializeField*. n.d.-k. Unity Documentation. URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/SerializeField.html>.
- [53] Valve Corporation. *Hacknet*. n.d. Steam. URL: <https://store.steampowered.com/app/365450/Hacknet/>.
- [54] Valve Software. *About Valve*. n.d. URL: <https://www.valvesoftware.com/en/about>.
- [55] Henrik Warpefelt. 'The Non-Player Character: Exploring the Believability of NPC Presentation and Behaviour'. DiVA Portal. Doctoral dissertation. Stockholm University, Faculty of Social Sciences, Department of Computer and System Sciences, 2016. URL: <https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A912617&dswid=-1187>.

Appendices

1. Link to the GitHub repository - https://github.com/Deni4kaHun7/Cyber_Academy-BT?tab=readme-ov-file#readme
2. Link to play the game - <https://dseni4ka.itch.io/cyber-academy>
3. Link to the questionnaire used to test the testers' knowledge of phishing -https://docs.google.com/forms/d/e/1FAIpQLScF18D_iK1Mbx-ixZT8SZ0YFGA011q59Wh2QrUM5v7p_e80Uw/viewform?usp=sharing